



Bachelorarbeit

Grid und Torusbasiertes Fast Failover Routing

Sebastian Peters

22.06.24

Begutachtung:
Prof. Dr. Dr. Klaus-Tycho Förster
Stephanie Althoff, M. Sc.

Abstract

In this thesis, grid and torus-based fast failover methods are explored to enhance the resilience of network infrastructures. The focus is on software-defined networking (SDN) environments, where rapid recovery is crucial. Several failover strategies are evaluated, with particular attention to a low-stretch algorithm and a Hamiltonian-based method. Various failure patterns are employed to simulate multiple failure scenarios. The implementation is carried out entirely in OpenFlow, utilizing fast failover groups. The results confirm that the low-stretch algorithm provides superior throughput and latency under failure conditions due to its efficient routing structure. It should be emphasized that the low stretch algorithm works even better than its theoretical resilience guarantees.

Inhaltsverzeichnis

Abstract	i
1. Einführung	1
1.1. Motivation und Hintergrund	1
1.2. Beitrag	1
1.3. Struktur der Arbeit	1
2. Grundlagen	3
2.1. Gegenwärtige Netzwerkinfrastruktur	3
2.1.1. Software-definiertes Networking	3
2.1.2. OpenFlow	4
2.2. Ausfallszenarien bei Netzwerkinfrastruktur	4
2.3. Resilienz schaffen	5
2.3.1. Traditionelle Lösungen	5
2.3.2. Fehlertoleranz für SDNs	5
2.3.3. Fast Failover	6
2.4. Modell und Topologie: Ein Torus	7
3. Verwandte Arbeiten	9
3.1. Spezifisch verwandte Arbeiten	9
3.1.1. »Exploring the Limits of Static Resilient Routing«	9
3.1.2. »Local Fast Failover Routing With Low Stretch«	10
3.1.3. »Perfect Resilience with Local Fast Failover«	11
4. Methodik und Implementation	13
4.1. Netzkemulation	13
4.1.1. Mininet	13
4.1.2. OpenVSwitch	13
4.2. Methodik: Ausfallmuster und Traffic Pattern	14
4.3. Methodik: Erfassung von Messwerten	15
4.3.1. Erreichbarkeit und Latenz	16
4.3.2. Durchsatz	17
4.3.3. Einschränkung und Erwartung	18
4.4. Architektur der Implementierung	19
4.4.1. Implementierung der Fastfailover Algorithmen und Augmentation	20
4.4.2. Implementierung: Hamilton Zykle Fast Failover	21
4.4.3. Implementierung: Low Stretch Fast Failover	23
4.4.4. Implementation: Perfect Resiliency within 2-Hops	24

Inhaltsverzeichnis

5. Evaluation	27
5.1. Latenz und Erreichbarkeit	27
5.1.1. Failure pattern: Random Edges	27
5.1.2. Failure pattern: Random Nodes	29
5.1.3. Failure pattern: Clustered Failiures	31
5.2. Durchsatz	34
5.2.1. Failure pattern: Edges	34
5.2.2. Failure pattern: Randome Nodes	34
5.2.3. Failure pattern: Clustered Failure	34
5.2.4. Failure pattern: Towards Destination	35
5.3. Augmentation: Perfect Resiliency within 2-Hops	38
6. Diskussion	41
7. Abschluss	43
7.1. Rekapitulation	43
7.2. Ausblick	43
7.3. Fazit	44
Literaturverzeichnis	51
Abbildungsverzeichnis	52
A. Anhang Preliminary testing	53
A.1. Latenz	53
A.2. Durchsatz	53
A.3. Cluster Failure Parametrisierung	55
B. Reproduzierbarkeit der Forschungsergebnisse	56

1. Einführung

In diesem Kapitel wird zuerst die Motivation und der Hintergrund (Sektion 1.1) der Arbeit kurz skizziert und abschließend die Zielsetzung und der Beitrag (Sektion 1.2) spezifiziert. Abschließend folgt eine kurze Übersicht über die Struktur (Sektion 1.3).

1.1. Motivation und Hintergrund

In der heutigen Zeit ist die Verfügbarkeit von Netzwerkinfrastrukturen von hoher Bedeutung. Ausfälle sind weitverbreitet, wodurch erhebliche Auswirkungen auf Infrastrukturen und Dienste entstehen können. Moderne Netzwerke in Datacentern verfügen zwar häufig über schnelle lokale Wiederherstellungsmechanismen, auch bekannt als Fast Failover, wegen ihrer lokalen Anwendung bestehen jedoch erhebliche Einschränkungen hinsichtlich der verfügbaren Informationen und der Fähigkeiten der ausführenden Netzwerkkomponenten. Backupwege erweisen sich oft als suboptimal, und es können keine theoretischen Garantien gegeben werden, die über den Ausfall weniger Verbindungen hinausgehen. Einhergehend mit der (relativen) Konfigurationsfreiheit von SDNs und OpenFlow lassen sich jedoch erstmalig erweiterte Verfahren direkt konfigurieren. Des Weiteren ist nach der Forschung von Foerster et al.[20] doch ein Verfahren für die Topologie eines Torus mit optimaler Weglänge (für ein Fast Failover Verfahren) bekannt.

1.2. Beitrag

Der Beitrag dieser Arbeit besteht in der Konzeption und Implementierung eines Frameworks, um torusbasierte Fast Failover Algorithmen unter verschiedenen Metriken in Mininet zu evaluieren. Dabei werden differenzierte Fehlermuster berücksichtigt. Die Realisierung erfolgt vollständig in OpenFlow mithilfe von Fast Failover Gruppen, um eine (noch) praxisnähere Evaluation zu ermöglichen. Zur Portfolio der Algorithmen gehört insbesondere ein Low Stretch[20] Verfahren und ein Hamilton Zyklus basierendes Verfahren[11], das im Rahmen dieser Arbeit modifiziert wird, um initial opportunere Route zu bevorzugen. In der Zielsetzung wird daher das Verhalten von beschriebenen Fast Failover Verfahren unter verschiedener Fehlermustern ausgewertet, in Umsetzung der mit OpenFlow Fast Failure Gruppe. Es ist auch zu ergründen, in welchem Ausmaß das Resilienzverhalten außerhalb theoretischer Garantien divergiert.

1.3. Struktur der Arbeit

Innerhalb des Grundlagenteils und der verwandten Arbeiten (Kapitel 2 und 3) werden die Grundlagen der Thematik des Fast Failover und der Topologie eines Torus vermittelt und relevante Forschung beleuchtet, insbesondere die Publikation von Foerster et al.[20, 18] sowie Chiesa et al.[11] mit den untersuchten Fast Failover Verfahren. Weiter wird im Teil der Methodik und Implementierung (Kapitel 4) die Methodik der Evaluation erläutert und die Architektur der Implementierung ausgeführt. Nachfolgend werden innerhalb

1. Einführung

der Evaluation (Kapitel 5) in Zusammenschluss mit der Diskussion (Kapitel 6) Resultate der Evaluation der Fast Failover Verfahren präsentiert und eingeordnet. Zuletzt wird ein Ausblick auf zukünftige Forschung und Ergebnisse dieser Arbeit im Abschluss (Kapitel 7) gegeben.

2. Grundlagen

In diesem Kapitel innerhalb der Sektion 2.1 wird die aktuell verwendete (Netzwerk-)Infrastruktur beleuchtet, mit besonderem Fokus auf Software-definiertes Networking. Anknüpfend werden Ausfallszenarien (Sektion 2.2) elaboriert und Lösungskonzepte zum Schaffen von Resilienzen präsentiert (Sektion 2.3). Schließlich folgt eine Beschreibung der Topologie eines Torus und eine Spezifizierung des verwendeten Fast Failover Modells in dieser Arbeit (Sektion 2.4).

2.1. Gegenwärtige Netzwerkinfrastruktur

Die Netzwerkinfrastruktur bildet einen essenziellen Eckpfeiler des täglichen Lebens und der wirtschaftlichen Aktivität in einer global vernetzten Welt. Breitgefächert umfasst diese Infrastruktur eine Vielzahl von Technologien, Plattformen und Protokollen, die es ermöglichen, Daten zwischen verschiedenen Geräten, Standorten und Organisationen zu übermitteln und zu verarbeiten. Folgerichtig ergeben sich mit zunehmender Größe und immer wachsendem Datenvolumen immer komplexere Anforderungen an gerade die mittelbare Infrastruktur[72]. Es ist notwendig, flexibel auf rapide Veränderungen der Anforderungen zu reagieren, um die Zuverlässigkeit des Netzwerks unter Einhaltung eines möglichst geringen Administrationsaufwandes sicherzustellen. Daher ist es opportun, Störungen alsbald, mit möglichst geringer Verzögerung zu korrigieren und Konfigurationen bevorzugt zu automatisieren. Historisch waren die Funktionalität des Routing und der Konfiguration der einzelnen Netzwerkkomponente in sich vereint. Demnach waren Anpassungen an der Konfiguration oft nur durch direkte Modifikation an den Netzwerkkomponenten möglich. Plausibel ist dies der verlangten Flexibilität und Skalierbarkeit abträglich[62, 61].

2.1.1. Software-definiertes Networking

Software-definiertes Networking (SDN) soll indes durch Trennung von Kontrolle und Datenebene Abhilfe schaffen. So beschreiben Stallings[61] ferner, dass die Datenebene nun die aktive Verantwortung der Weiterleitung der Pakete auf den Switches trägt, wenngleich die Kontrollebene als zentrale Instanz die Routen an den Anforderungen konfiguriert. Im Detail offenbart der physikalische Switch eine virtuelle Schnittstelle, die von dem zentralen Controller manipuliert wird. Dies wird im Allgemeine als »South-bound API« bezeichnet und in der Regel über einen Standard wie OpenFlow vereinheitlicht (dazu sei auf Abschnitt 2.1.2 verwiesen). Es obliegt dennoch dem Hersteller, jene Standards zu implementieren. Hierneben sei noch erwähnt, dass der Controller im Grundsatz über eine Datenanbindung zu jedem Switch verfügt und somit über eine globale Ansicht des Netzwerkes. Es kann daher flexibel auf veränderte Rahmenbedingungen reagiert und nuancierter nach QoS (Quality of Service) Routen konfiguriert werden. Gleichwohl sind die Fähigkeiten des eigentlichen Switches bedingt durch den gewählten Standard und die Konstruktion limitiert. Im Kontrast dazu ist die Software des Controllers nur an die Fähigkeiten der Datenebene gebunden und so frei in der Verwirklichung.

2. Grundlagen

2.1.2. OpenFlow

OpenFlow[50, 43] ist ein Protokoll, das die Konfiguration zwischen dem zentralen SDN-Controller und den Netzwerkgeräten wie Switches und Routern standardisiert. Die Funktionsweise basiert auf dem Konzept sogenannter Flowrules, die das Headerfeld eines Datenpaketes anhand einer Vielzahl von Bedingungen einer oder mehrerer Aktionen zuordnen. Diese Flowrules werden in Form von Einträgen in sogenannten Flow-Tabellen des Netzwerkgeräts gespeichert. Die Zuordnung des Datenpakets erfolgt dann in der Regel basierend auf Kriterien wie Quell- und Ziel-IP-Adressen oder Art des Datenpakets. Allerdings sind auch exotischere Fallunterscheidungen möglich.

Aktionen können die Weiterleitung an einen Ausgangsport, an eine andere Flow-Tabelle oder an eine Gruppe beinhalten, aber auch die Modifikation, die Übermittlung an den Controller zur Inspektion oder das Verwerfen des Pakets. **Gruppen** hingegen stellen eine Möglichkeit dar, das Weiterleitungsverhalten nuanciert zu steuern und Funktionalitäten wie Broadcasts, Load Balancing und Fast Failover zu verwirklichen.

Anwendungsbezogen zur Thematik der Arbeit führt die »**Fast Failover Gruppe**« den ersten lebendigen »Bucket« aus, wobei ein Bucket in der Regel mit einer Ausgabeaktion zu einem Ausgangsport korrespondiert. Die Lebendigkeit eines Bucket wird hingegen über die Bereitschaft eines spezifizierten (Out-)Ports bestimmt[50]. Zu akzentuieren ist, dass dies ohne Einsatz des Controllers vonstattengeht und so die Kerndefinition des Fast Failover erfüllt.

2.2. Ausfallszenarien bei Netzwerkinfrastruktur

Ausfallszenarien sind oft multikausal und können mitunter folgenschwer sein, wenngleich auch ihre Einsatzfrequenzen und Auswirkungen stark variieren. Mikroökonomisch bietet sich folgend der Arbeit Turner et al.[68], in der die Autoren über mehrere Jahre hinweg die Ausfälle des staatenweiten CENIC-Netzwerks analysieren, ein differenziertes Bild. So sind eine geringe Anzahl von Verbindungen für mehr als Hälfte der Verbindungsausfälle verantwortlich. Als Ursachen werden mehrheitlich Software- und Hardwarefehler angegeben, allerdings resultierten Hardware- und Energieprobleme in den längsten Reparaturzeiten. Externe Beeinträchtigungen ließen sich größtenteils durch Reparaturmechanismen abwenden. In Gänze ist (Netzwerk-)Hardware grundsätzlich als zuverlässig anzusehen. Dieses Ergebnis deckt sich mit der Forschung von Meza et al.[45].

Makroökonomisch ergibt sich ein etwas anderer Eindruck[22]. Obzwar (fatale) Ausfälle dieser Schwere selten sind, kann eine Störung des Endnutzerdienstes erhebliche Kosten von mehreren Millionen Dollar verursachen[37, 71]. Die genannten Zahlen sollten jedoch mit Vorsicht betrachtet werden, da ihre Quellen potenzielle Interessenkonflikte aufweisen. Gleichmaßen sind auch hier die Ursachen häufig facettenreich: von einfachen Bugs über Konfigurationsfehler bis zu Naturkatastrophen oder Überlastungen durch zu viel Traffic. Gemeinsam ist häufig nur, dass auch Wiederherstellungsmaßnahmen nicht einsetzten oder versagten und als Folge Dienste nicht verfügbar waren oder Daten verloren gingen, mit den entsprechenden folgenschweren Konsequenzen.

Schließlich darf nicht unerwähnt bleiben, dass die Erkennung und Lokalisierung von Ausfällen nicht unbedingt trivial ist. Physikalische Beschädigungen eines Lichtwellenleiters können ggf. von dem Switch direkt am Ausbleiben des Lichtsignals detektiert werden[52]. Komplexere Szenarien bedingen aufgrund der diffusen Fehlerbilder oft aufwändigere Maßnahmen[56, 76]. So können exemplarisch Netzwerkkomponenten nur teilweise ausfallen und geringe Paketverluste erzeugen oder die Ursache ist von außen nicht

ersichtlich. Letztlich basiert die Detektion auf Auswertung der Parameter des Netzwerkes zu Anomalien oder auf Probing. Praktisch unterscheidet sich auch die Zeit der Erfassung von einigen Millisekunden (im ersten beschriebenen Fall) bis zu mehreren Sekunden im Falle von IGP[60] bei ungünstig gewählten Parametern.

2.3. Resilienz schaffen

Solange Störungen (unvermeidlicherweise) auftreten, ist es nur sachdienlich, die Folgen eines Ausfalls zu begrenzen. Häufig liegt der Heilsbringer in der Wiederherstellung der Funktionalität des Dienstes[44, 35]. Gleichwohl erlauben unterschiedliche Szenarien differenzierte Wiederherstellungsmaßnahmen, die sowohl in Wiedererstellungsgeschwindigkeit als auch Toleranz und Opportunität variieren. Im anknüpfenden Abschnitt werden verschiedene Möglichkeiten der Resilienz erläutert, mit besonderem Fokus auf das Fast Failover Routing als Subjekt dieser Arbeit. In Gänze wird sich aufgrund des Umfanges auf in-Data Center Kommunikation beschränkt.

2.3.1. Traditionelle Lösungen

Auch in traditionellen Netzwerkarchitekturen nach Khan et al.[35] können Maßnahmen der Resilienz geschaffen werden. Maßnahmen gegenüber isolierten Anbindungsausfällen lassen sich durch Aggregation schaffen (bspw. ECMP[29]). Backup Hardware kann beim Ausfall einzelner Knoten durch Hot Swap Protokolle wie VRRP[48] oder HSRP[39] im laufenden Betrieb eingesetzt werden. Weiterhin können Dynamische Routing Protokolle wie OSPF[46] Ausfälle durch neue Berechnung und Verteilung der Routingtabellen kompensieren oder Verfahren wie RSTP[25] genutzt werden, um auf redundante Wege umzuschalten. Zuletzt bleibt dem Netzwerkadministrierenden die manuelle Rekonfiguration als Notanker. Die Recoveryzeiten reichen hier von wenigen Millisekunden (ECMP [29]) bis zu 30 Sekunden (Link State) oder länger im Falle eines Hot Swaps. Die lange Wiederherstellungszeit ist beispielsweise im Falle von OSPF [46] durch die Funktionsweise eines Link State Protokolls begründet, da Änderungen der Routingtabellen (exemplarisch ausgelöst durch eine Modifikation der Topologie durch Ausfall) erst über das gesamte Netzwerk propagiert werden müssen. Andernfalls ist die Identifizierung eines Ausfalls initial langwierig.

2.3.2. Fehlertoleranz für SDNs

Bei Störung von SDN Infrastruktur wird nach der Publikation von Menaceur et al.[44] im Allgemeinen zwischen Ausfällen der Datenebene und der Kontrollebene differenziert. Auf der Datenebene kann der Controller aufgrund seiner globalen Übersicht bei einem Ausfall reaktiv alternative Routen einrichten, ohne den intakten Datenfluss zu unterbrechen. Hingegen stellt die Kontrollebene eine Herausforderung dar, denn eine Unterbrechung der Kommunikation zwischen Controller und Netzwerkkomponenten kann ein konkretes Betreiben der Netzwerkinfrastruktur unmöglich machen. Allerdings ist auch hier Abhilfe möglich. Gegen Ausfall des Controllers lassen sich Redundanzen einrichten und konträr zu einer Verbindungsunterbrechung können Anbindungen zum Controller mehrfach angelegt werden. Fortwährend kann das Netzwerk proaktiv durch Installation unabhängiger (Re-)Routing Mechanismen wie RSTP[25] oder Fast Failover Routing Verfahren ertüchtigt werden, auch autark vom Controller zu agieren. Die Recovery bzw. Konvergenz Zeiten variieren ebenfalls, von wenigen Sekunden im Falle von Rerouting des Controllers bis zu mehreren Minuten im Falle eines (ungünstigen) Hot Swaps.

2. Grundlagen

2.3.3. Fast Failover

Auch wenn die dargestellten Verfahren eine Wiederherstellung ermöglichen, kann die erzeugte Unterbrechung möglicherweise nicht den zeitlichen Anforderungen des Dienstes genügen. So gehen bei einem 10-Gigabit-Link während des Ausfalls von einer Sekunde bereits ca. 833k Pakete mit einer Größe von jeweils 1500 Bytes verloren[17]. Um Abhilfe zu schaffen, gibt es eine Reihe von Verfahren, die deutlich schnellere Recovery-Times ermöglichen und in der Regel direkt auf der Datenebene angesiedelt sind. Weiter charakterisiert Chiesa et al.[12] die als Fast-Reroute (FRR) oder Fast-Failover bekannten Verfahren als Mechanismen, die vorberechnete Routen nutzen, um im Falle eines Ausfalls Pakete entsprechend umzuschalten. Dementsprechend obliegt dem Controller im Falle des SDNs lediglich die (vor-)Berechnung jener Routen. Als prominente Vertreter dieser Gruppe sind noch zu MPLS fast Reroute[4, 36], Openflow-fast-Failover-Group[50] oder simples ECMP[29] nennen. Häufig werden FRRs als erste Linie der Verteidigung betrachtet, die eine schnelle, aber suboptimale Wiederherstellung ermöglichen, während subsektiv ein anderes Verfahren mit möglicherweise besseren Routen übernimmt. Aus den nur eingeschränkten Fähigkeiten der Datenebene und nur lokal verfügbaren Informationen ergeben sich zusätzliche Herausforderungen für die Konstruktion eines Fast Failover-Verfahrens. Resultierend können auf den meisten Topologien nur eine begrenzte Anzahl an Ausfällen toleriert werden.

Es sei erwähnt, dass dies konkret auf die nachfolgend im Text beschriebenen statischen deterministischen Verfahren zutrifft und nicht notwendigerweise auf anderen FRRs beispielsweise mit dynamischen Tabellen. Im Detail, so führt Chiesa et al.[13] weiter aus, werden Routingtabellen als statisch betrachtet, sofern keine dynamischen oder adaptiven Veränderungen bei einem Ausfall oder einer anderen Bedingung erfolgen. Diese Abgrenzung ist wesentlich, da nicht statische Tabellen bereits Toleranz gegen eine beliebige Anzahl an Ausfällen erlauben, sofern der Graph noch zusammenhängend ist (auch bezeichnet als perfekte Resilienz[17, 18]), vor allem, da jene kaum auf aktueller Hardware im Fast Failover realisiert werden können. Im Weiteren werden daher nur lokal statische Verfahren betrachtet, die ihr Weiterleitungsverhalten anhand des Paket-Headers und dem Verbindungsstatus der direkt benachbarten Kanten ausrichten. Eine Modifikation (Tagging) oder Duplikation des Paketes ist in diesem Kontext ausgeschlossen, auch wenn dies nachweisbar die Resilienz anheben kann[11, 13] (etwa durch Annotation aller ausgefallenen Kanten wird eine globale Übersicht erzielt oder durch Duplikation wird das Paket über das ganze Netzwerk geflooded). Dies ist eventuell inopportun aufgrund des induzierten Durchsatzes, des Unvermögens der Hardware oder anderer analoger Faktoren wie Loops[12]. Das Einbeziehen von Zufallsquellen in die Routing-Entscheidungen kann zwar eine perfekte Resilienz ermöglichen (etwa durch gleichmäßig zufällige Wahl einer Kante), geht jedoch mit längeren Backuppfaden und dem Verlust des Determinismus einher[8, 13]. Insgesamt kann nach Foerster et al.[18, 19] resümiert werden, dass perfekte Resilienz unter den ausgeführten Einschränkungen selbst für dichte Topologien (einschließlich initial vollständiger Graphen) unerreichbar ist, mit der Ausnahme einer gewissen Anzahl von Graphklassen und ihren Derivaten.

Eine schwächere Aussage lässt sich für nach Chiesa et al.[13] für eine Reihe an k -connected Topologien treffen, obgleich der Beweis für $k > 5$ bei allgemeine k -connected Topologien bis dato. aussteht[19]: nach Chiesa et al.[13][S.2], »Conjecture: For any k -connected graph, one can find deterministic failover routing functions that are robust to any $k - 1$ failures.«. Ein solches Verfahren wird auch als ideale (eng. *ideal*) Resilienz beschrieben[19].

Schließlich werden Verfahren noch über den Umfang des Matchings über den Packet-Header differenziert[13], denn mit zunehmender Größe der Routingtabelle steigt ihre Komplexität. So ergibt sich *par exemple* eine quadratische Abhängigkeit zur Größe der Tabelle, wenn jedem Source-Ziel-Tupel ein Eintrag zugeordnet

wird.

Als Randnotiz ist aber im Besonderen noch das Konzept der Spanning Tree Arborescences hervorzuheben[13] basierend auf der Zerlegung eines Graphen r -rooted spanning arborescence. Hierbei handelt es sich um k kantendisjunkte Spannbäume gewurzelt von der Destination. Präzisierend ist »kantendisjunkt« allein nicht ganz treffend, denn es wird nur eine disjunkte (Kanten-)Menge bezogen auf eine Richtung einer Kante gefordert, in der Literatur dann auch bekannt als »Arc«. In Übertragung auf ein Fast Failover Verfahren kann abstrakt, sofern eine Arborescences ausgefallen ist, auf eine andere gewechselt werden, wobei die Entscheidung nicht unbedingt trivial sein muss.

2.4. Modell und Topologie: Ein Torus

Als Modell – übernommen aus Foerster et al.[20] – wird ein ungerichteter Graph $G = (V, E)$ betrachtet, in dem Kanten als Verbindungen und Knoten als Netzwerkschwitches realisiert sind. Zusätzlich kann ein zu einem Knoten adressiertes Paket an einem (anderen) beliebigen Knoten eingebracht werden. Divergierend der Definition aus Foerster et al. werden Kantenausfälle immer beidseitig angenommen. Ebenso bestimmt jeder Knoten anhand einer statischen Forwarding-Funktion basierend auf Eingangskante, Zieladresse und der Menge der ausgefallenen Kanten, die zur Weiterleitung bestimmte Kante. Herauszustellen ist die Analogie zu Openflow mit der Fast Failover Gruppe, die sich jedoch in Verfügbarkeit der Informationen der Ausgefallenen-Kanten unterscheidet. In Einzelheiten bedingt durch die Processingpipeline von Openflow findet die Gruppe erst Anwendung, wenn die (ingress) Flowtables durchlaufen sind[50].

Im Folgenden wird nunmehr der Aufbau der Topologie beschrieben. Geometrisch entsteht ein Torus durch Drehung eines Kreises um eine zum Kreis koplanare Achse[34]. Mittelbar entsteht das gleichnamige Netzwerk auf der Oberfläche dieses Torus. Dies stellt im zweidimensionalen Fall ein Netzwerk dar, in welchem die Knoten auf einem Gitter angeordnet sind, wobei die äußeren Knoten zusätzlich mit ihrem Pendant auf der gegenüberliegenden Seite verbunden sind. Anschaulich lässt sich ein solches Netzwerk an einem Gitter aufgespannt über die Fläche eines »Donuts« illustrieren. Im weiteren Verlauf wird ausschließlich die zweidimensionale Ausprägung referenziert. Als topologische Eigenschaften sind insbesondere die Vertex-Transitivität und die Connectivity of four anzuführen[63]. Erstere impliziert, dass es für jeden Knoten einen Automorphismus gibt, der ihn auf einen beliebigen anderen Knoten abbildet. Dies ist für die praktische Konstruktion von Routing Algorithmen hilfreich, da jeder Knoten des Netzes homogen ist und derselbe Routing-Algorithmus auf jeden Knoten angewendet werden kann. Überdies resultiert die Connectivity in vier Kanten-disjunkten Pfaden zwischen allen paarweise verschiedenen Knotenpaaren, sodass es wenigstens vier Kantenausfälle erfordert, um den Graphen zu zertrennen.

Zuletzt bleibt die kurze Diskussion bezüglich der Grenzen des Fast Failover Routing resultierend aus der Topologie. Bei der Topologie handelt es sich um einen four-connected Graph, entsprechend ist eine Resilienz von drei für deterministisch statische Fast Failover Verfahren möglich[14, 18, 20]. Im Kontrast dazu ist die perfekte Resilienz ausgeschlossen, denn es handelt sich um einen nicht planaren Graphen, der nach Foerster et al.[Thm 4.4][18] keine perfekte Resilienz erringen kann.

3. Verwandte Arbeiten

Innerhalb dieses Kapitels werden zunächst verwandte Forschungen, insbesondere zum Thema Fast Failover, betrachtet. Anschließend folgt der Abschnitt der spezifisch verwandten Arbeiten (Sektion 3.1) mit einer Aufschlüsselung besonders relevanter Publikationen für diese Arbeit.

Einleitend mit der Publikation von Feigenbaum et al.[17]. Zusammenfassend wird zusätzlich zu relevanter Grundlagenforschung ein (statisches) Fast Failover Verfahren präsentiert, das zum ersten Mal für allgemeine Topologien eine Resilienz von eins ausweist. Ferner wird das Paper von Borokhovich and Schmid[8] diskutiert, in dem die Autoren Fallstricke in Bezug auf das Fast Failover Routing analysieren und speziell den Link-Load im Zusammenhang mit Kantenausfällen untersuchen. Weiterhin werden die Möglichkeiten eines SDN Netzwerkes ergründet und mehrere Verfahren angegeben. Anschließend beschreibt die Veröffentlichung von Schweiger et al.[59] die Grundlage für ein in dieser Arbeit verwendetes Failure Pattern (Sektion 4.2). Abseits dessen werden zwei Verfahren unter Ausnutzung von Edge-Disjoint Paths elaboriert und evaluiert. Zu den Grenzen der perfekten Resilienz zeichnet Foerster et al.[19] ein umfassendes Bild.

3.1. Spezifisch verwandte Arbeiten

Innerhalb dieses Abschnitts werden die spezifischen verwandten Publikationen betrachtet, die eine besondere Bedeutung für diese Arbeit haben. In den ersten beiden Subsektionen (3.1.1 und 3.1.2) werden die Arbeiten vorgestellt, die Fast Failover Algorithmen für die Evaluation bereitstellen. Nachfolgend wird die Forschung vorgestellt, die Basis für eine Augmentation der Algorithmen beschreibt (Subsektion 3.1.3).

3.1.1. »Exploring the Limits of Static Resilient Routing«

Die Publikation von Chiesa et al.[11] ist von besonderer Relevanz für diese Arbeit, da die Autoren einen deterministischen Failover-Algorithmus für Torus Topologien präsentieren, der auf Hamilton-Zyklen basiert. Abseits dessen liefert die Arbeit wichtige Resultate bezüglich der Grenze des Fast Failover Routings unter unterschiedlichsten Befähigungen der Switches. Dazu werden mehrere Verfahren präsentiert, insbesondere werden auch statisch deterministische Algorithmen für allgemeine k -connected Topologien bis zur Resilienz von $k = 5$ beschrieben. Jene weisen eine Resilienz von $k - 1$ aus.

Überleitend zum Algorithmus: Ein Hamilton-Zyklus ist ein geschlossener Pfad, der jeden Knoten eines Graphen genau einmal durchläuft[69]. Die Funktionsweise des Algorithmus lässt sich wie folgt beschreiben: Zunächst wird das Paket entlang des ersten Hamilton-Zyklus weitergeleitet, bis es sein Ziel erreicht oder auf eine fehlerhafte Kante stößt. Trifft es nun auf eine fehlerhafte Kante, wird es entsprechend dem ersten Hamilton-Zyklus zurückgesendet. Sollte nun wieder eine ausgefallene Kante auf dem direkten Weg liegen, so wird mit dem zweiten Hamilton-Zyklus analog verfahren. Die zwei Zyklen sind in Abbildung 3.1 jeweils in Rot und grau schraffiert für Tori mit ungerader Dimension (bezogen auf die

3. Verwandte Arbeiten

Knoten) dargestellt. Das Muster lässt sich durch Hinzufügen von zwei Reihen an den Markierungen fortsetzen. Weiterhin sei für Tori mit geraden Dimensionen auf Chiesa et al.[11] verwiesen. Die Ermittlung des aktuellen Hamilton-Zyklus und seiner Laufrichtung kann über die Eingangskante des Knotens erfolgen. Zur Bilanz zeigt sich zwar eine ideale Resilienz, aber ein suboptimaler Stretch mit $\Omega(n)$ [20]. Stretch bezieht sich hierbei auf die Länge der Routen zwischen zwei Zielen, die im Falle von Kantenfehlern im optimalen Fall ebenfalls minimal sein sollten. Ebenso ist zu bemerken, dass allgemein aus mehreren kanten-disjunkten Hamilton Zyklen auch für andere Topologien ein derartiger Failover Algorithmus konzipiert werden kann[11].

3.1.2. »Local Fast Failover Routing With Low Stretch«

Fortführend die Forschung von Foerster et al.[20]. Diese ist maßgeblich für diese Arbeit aufgrund des beschriebenen deterministischen Fast Failover Verfahrens für Tori, das ideale Resilienz gegenüber Kantenfehlern mit optimalem Stretch zusichert. Zusätzlich beschreibt die Veröffentlichung weitere Stretch optimale Verfahren für elementare Topologie wie multidimensionale Grids oder Clos Netzwerke. Bei letzterem handelt es sich um oft verwendete Topologien innerhalb von Datenzentren oder Clustern.

Im Anschluss wird das Verfahren kurz umrissen und anschließend vollständig charakterisiert. Konzeptuell konstruiert der Algorithmus einen Umweg um die ausgefallene Kante mithilfe der Geometrie des Torus. Genauer gesagt wird das Netzwerk in vier Quadranten unterteilt, wobei der Zielknoten als Mittelpunkt und die Koordinatenachsen als Trennlinien dienen. Wenn ein Paket auf eine gestörte Kante trifft, wird es in einen anderen Quadranten weitergeleitet, wobei die Topologie in den meisten Fällen trotzdem eine Distanzreduzierung zum Ziel bedingt. Im Detail wird der Stretch bei einer ausgefallenen Kante nur maximal um zwei erhöht bzw. $2f$ im Falle der Störung mehrerer Kanten.

Es folgt nun die vollständige Charakterisierung des Failover Routing Schemas für zweidimensionale Tori. Beginnend ohne Verlust der Allgemeinheit wird ein Modell angenommen, indem alle Knoten zu einem Ziel kommunizieren. Dies impliziert zunächst keine Einschränkung, da ein derartiges Schema unter Identifikation des Zieles für beliebige Knoten des Torus einzurichten wäre. Weiter sei der Zielknoten bezeichnet als t innerhalb des zwei dimensional Koordinatensystems mit x_1 und x_2 Achse. Zusätzlich sei t als Mittelpunkt im Ursprung fixiert. Gleichermäßen ist dies kein Hindernis, da unter Ausnutzung der zyklischen Eigenschaften des Torus eine Projektion für einen beliebigen Knoten erfolgen

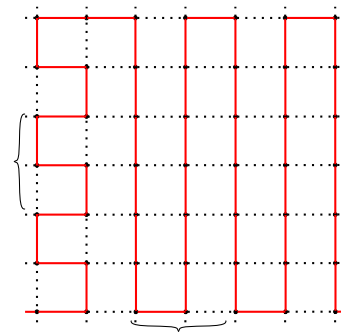


Abbildung 3.1.: Hamilton Zyklen in einem Torus mit ungeraden Dimensionen. Aus [11].
Zyklus 1: rot; **Zyklus 2:** schraffiert grau

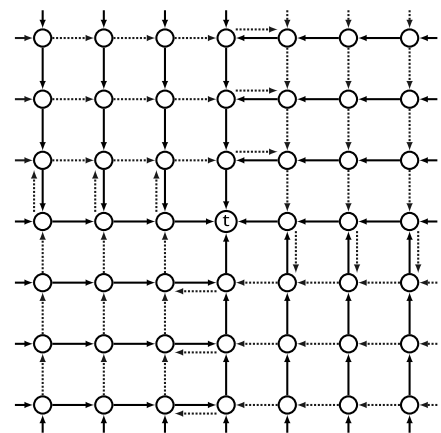


Abbildung 3.2.: Aus [20]. e_t Regeln sind gefüllt. e_l Regeln sind gestrichelt.

kann. Daraufhin sei das Koordinatensystem in vier Quadranten eingeteilt, zerteilt durch die Achsen, die mithilfe des Vorzeichens der x_1 - und x_2 zu identifizieren sind. Anschließend kann ein Shortest-Path-Routing-Tree nach folgendem Schema konstruiert werden:

Algorithm 1 Konstruktion des shortest-path-routing-tree for ein Ziel t aus [20]

Require: $v \in V$

```

if Quadrant( $v$ ) = ++ then decrease  $x_1$  to 0, then decrease  $x_2$ 
if Quadrant( $v$ ) = -+ then decrease  $x_2$  to 0, then increase  $x_1$ 
if Quadrant( $v$ ) = -- then increase  $x_1$  to 0, then increase  $x_2$ 
if Quadrant( $v$ ) = +- then increase  $x_2$  to 0, then decrease  $x_1$ 

```

Als Randnotiz zur Konstruktion sei auch auf Abbildung 3.2 verwiesen. Insbesondere ist jeder Knoten, mit Ausnahme von t (im Mittelpunkt) einem Quadrat zugeordnet. Das Ziel der ausgehenden Kanten wird nach dem konstruierten Baum für jeden Knoten als $e_t(v)$ notiert. Anschließend ausgerichtet an $e_t(v)$ werden linksseitig $e_l(v)$ und rechtsseitig $e_r(v)$ sowie zur gegenüberliegenden Seite $e_b(v)$ festgehalten. Schließlich lassen sich die Routingregeln konkretisieren:

Algorithm 2 Weiterleitung Regeln des Low Stretch Fast Failover aus [20]

Require: $v \in V$, $e_x(v)$ konstruiert für t

```

if  $e_t(v)$  is up  $\wedge$  in_port  $\neq e_t(v)$  then Output:  $e_t(v)$ 
if  $\exists w \in V$  mit  $(x_1(w) = 0 \vee x_2(w) = 0) \wedge e_l(w) = v$  then
    if in_port= $e_l(v)$  then Output:  $e_b(v)$ 
if Sonst then Output:  $e_l, e_b, e_r$ 

```

Fortsetzend eine kurze Beschreibung zur Mechanik unter Ausfällen. Vorher sei hervorgehoben, dass das Routing grundsätzlich zunächst anhand des konstruierten Baumes erfolgt (e_t Kante) und somit auf einem kürzesten Weg. Im Falle eines Ausfalles von e_t wird stattdessen e_l frequentiert und weiterhin die Wegstrecke verkürzt, sofern der Knoten nicht auf einer Achse liegt (dann werden zusätzlich 2 Hops benötigt). Dezidiert finde im gleichen Fall durch den Vergleich des Eingangsportes in der ersten Regel auch keine Rückleitung statt. Die weiteren Kanten e_r sowie e_b werden nur in Konstellation von drei Ausfällen am vorausgehend oder direkten Knoten eingesetzt. Letztlich erfolgt unter drei Ausfällen der Wechsel in einen freien der vier Quadranten und somit die Weiterleitung zum Ziel. Die zweite Regel verhindert einen Rückfall.

3.1.3. »Perfect Resilience with Local Fast Failover«

Die Forschung von Foerster et al.[18] ist zusätzlich substanziell für diese Arbeit, da Theorem 6.1 der Publikation die Basis für eine Augmentation der Algorithmen stellt. Gleichwohl leistet die Veröffentlichung darüber hinaus einen entscheidenden Beitrag in der Erforschung der perfekten Resilienz, insbesondere in der Beziehung zu verschiedenen Graphklassen unter anderem planare Graphen und Minors. Nachfolgend wird die Konzeption einer Augmentation aus Theorem 6.1 angeführt. In Aussage gibt Theorem [18, S.64]

3. Verwandte Arbeiten

ein Fast Failover Verfahren an, indem perfekte Resilienz über alle Wege errungen werden kann, sofern die Distanz zwischen Quelle und Ziel über den Weg noch zwei ist. Konzeptuell wird ein Paket an einen Nachbarn geleitet, der auf einem Weg der Länge zwei ist. Entweder wird das Paket anschließend zugestellt oder zurückgeleitet und die Quelle kann das Fehlschlagen des Weges identifizieren und einen anderen Nachbarn versuchen.

4. Methodik und Implementation

Innerhalb dieses Kapitels wird zuerst die verwendete Netzwerkemulation (Sektion 4.1) ausgeführt und anschließend innerhalb der Methodik (Sektion 4.3 und 4.2) die Erfassung von Messwerten sowie Traffic Pattern erörtert. Zuletzt folgt eine Beschreibung der Implementierung und des Test-Frameworks und der Verfahren (Sektion 4.4).

4.1. Netzwerkemulation

Auch die Disziplin der (Netzwerk-)Emulation ist intrinsisch facettenreich und es gibt eine Vielzahl an verschiedenen Lösungen, um den unterschiedlichen Anforderungen gerecht zu werden, wie auch Tsai et al.[67] in ihrer Veröffentlichung verschiedene Möglichkeiten aufzeigen. Dementsprechend kann diese Arbeit im folgenden Abschnitt lediglich einen Überblick über die verwendeten Technologien bieten, ohne den Anspruch der Vollständigkeit zu erheben.

4.1.1. Mininet

Mininet[23, 26] ist gemäß Handigol et al. ein containerbasierter Emulator, erwachsen aus dem Bedürfnis Forschungsergebnisse im Bereich, der Netzwerkforschung reproduzierbar zu machen. Die Verwendung von Containern in diesem Kontext impliziert, dass reale Anwendungen auf einem emulierten Netzwerk ausgeführt werden, wobei leichtgewichtig Virtualisierungstechniken auf Betriebssystemebene verwendet werden. Hierbei wird einer präzisen Ressourcenisolierung Rechnung getragen. Detailgenauer werden Linux-Container[38, 9] vergleichbar zu Jails in BSD[28, 55] verwendet, um Gruppen von Prozessen gegeneinander abzugrenzen und eigene Netzwerkkinterfaces anzubieten. Dies erfolgt jedoch auf demselben Kernel im Kontrast zur Vollsystemvirtualisierung. Für jeden virtuellen Host wird ein solcher Container erstellt, der wiederum an einen eindeutigen, jedoch unabhängigen Netzwerknamespace[15, 66] angehängt wird. Die virtuellen Interfaces werden schließlich mit Softwareswitches wie OpenVSwitch nach einer angeforderten Topologie verbunden und konfiguriert. Dabei lassen sich Charakteristiken realer Verbindungen über TC mit NetEm[27] simulieren. Ebenso steht eine Python-Schnittstelle zur Konfiguration zur Verfügung.

Unter der Prämisse dieser Arbeit darf auch nicht unerwähnt bleiben, dass es sich bei Mininet nicht um einen vollständigen Simulator handelt, da Mininet unter anderem an die Systemzeit gebunden ist. Dies kann bei Mangel von verfügbaren Ressourcen (CPU oder RAM) zu Diskrepanzen führen, begründet in der verzögerten Ausführung der meisten (seriellen) Events. Allgemein sind aber akkurate Ergebnisse zu erwarten, sofern es nicht zur Überlastung kommt[74].

4.1.2. OpenVSwitch

Als nächster Baustein wird nun OpenVSwitch[51] betrachtet. Bei OpenVSwitch handelt es sich laut Pfaff et al.[53] um einen State of the Art Softwareswitch, der entwickelt wurde, um den Bedarf der Netzwerkvir-

4. Methodik und Implementation

tualisierung speziell im Serverumfeld zu befriedigen. Konkret stellt OpenVSwitch daher eine performante und flexible Lösung mit voller Unterstützung von OpenFlow zur Konfiguration virtueller Netzwerke dar.

Architektonisch lässt sich OpenVSwitch in zwei Teile gliedern: das Kernelmodul (Datapath Kernel Module) und den Userspace (ovs-vswitchd). Das Kernelmodul ist naturgemäß hardwareabhängig und übernimmt das (direkte) Routing der Pakete, sofern bereits eine entsprechende »Aktion« konfiguriert ist. Ist dies nicht der Fall, wird das Paket an den Userspace weitergeleitet, der anschließend das Datapath-Modul entsprechend der konfigurierten Routen instruiert. Zur Beschleunigung wird Flow-Caching eingesetzt. Der Userspace-Daemon kann im Übrigen über einen SDN-Controller oder API konfiguriert werden.

Im Rahmen dieser Arbeit bietet sich eine Komposition aus Mininet und OpenFlow besonders an, da Mininet ein einfaches Prototyping auf normaler Hardware in hoher Qualität ermöglicht und OpenVSwitch als etablierte Software-Switch vollständig OpenFlow mit Fast Failover Gruppen unterstützt.

4.2. Methodik: Ausfallmuster und Traffic Pattern

Wie im Grundlagenteil bereits angerissen, sind Ausfallszenarien bei Netzwerkinfrastruktur inhärent komplex und in Teilen unvorhersehbar. Dennoch lassen sich Ausfallszenarien nach Manzano et al.[42] in zwei Kategorien unterteilen. Erstens die »Random« Ausfälle, die ihrem Namen entsprechend aufgrund eines willkürlichen Ereignisses auftreten, wobei eine Beschreibung durch statistische Modellierung jedoch möglich ist. Als zweite Kategorie werden »targeted« bzw. gezielte Ausfälle charakterisiert, welche maximalen Schaden unbestimmter Art erzeugen (sollen). Die nachfolgenden Ausfallmuster beschreiben eine zu deaktivierende Sequenz von Kanten, um ein möglichst breites Spektrum an Ausfallszenarien abzudecken. Die ersten drei fallen dabei unter die Kategorie des Zufalls und das letzte unter einen gezielten Angriff.

- **Random Edges:** Dieses Ausfallmuster wählt die zu deaktivierende Kante gleichverteilt zufällig aus.
- **Random Node:** Bei diesem Muster wird ein Knoten gleichverteilt zufällig ausgewählt und dessen adjazenten Kanten deaktiviert, sofern diese noch aktiv sind.
- **Clustred Failures:** In Bezugnahme auf die Forschung von Schweiger et al.[59] ist dieses Pattern dem Auftreten einer Naturkatastrophe mit einem Epizentrum nachempfunden, indem sich der Ausfall von einem oder mehreren Punkten auf mehrere Kanten propagiert. Konkret werden einer oder mehrere Punkte gewählt, um die sich der Ausfall mit degressiver Wahrscheinlichkeit per Hop auf die benachbarten Kanten ausbreitet, sofern jene Kante tatsächlich ausgefallen ist. Eingeordnet in den Kontext von Katastrophenereignissen könnte dies etwa auf Erdbeben oder lokale Stromausfälle zutreffen. Zusätzlich ist es auch nicht unüblich, dass Störungen kaskadiert auftreten[75, 73]. Wohingegen nicht unerwähnt bleiben darf, dass im Falle von Naturkatastrophen Disparitäten zu erwarten sind, denn in der Regel ist ein größerer geografischer Raum als Teil einer Datencenter-Topologie betroffen[49].

Modelliert wird die Ausbreitung über die Parameter d , p und f , wobei d die initialen Punkte oder auch »Epizentren« beschreibt. Zudem signifiziert p die primordiale Ausfallwahrscheinlichkeit der Kanten benachbart zu den Epizentren. Anschließend wird degressiv mit dem Faktor f auf die Propagationswahrscheinlichkeit eingewirkt. Zum genauen Einfluss verschiedener Parameter sei zusätzlich auf den Anhang A.3 verwiesen. Aus der Variation verschiedener Parameter im Rahmen der

4.3. Methodik: Erfassung von Messwerten

Evaluation resultieren verschiedene Anwendungsoptionen, um Einfluss auf den Umfang (Ausdehnung) bzw. Failure Rate zu nehmen: Im Detail werden einmal die Parameter p sowie q konstant gehalten und mehrere Punkte nacheinander eingebracht. Alternativ wird eine Menge von Punkten fixiert und p sowie q stufenweise innerhalb vorgegebener Intervalle modifiziert.

- **Towards Destination:** Orientiert sich an der Vorgehensweise von Borokhovich und Schmid[Clm. 4][8] sowie Foerster et al.[21]. Dabei werden die Kanten, kreiselnd um eine erwählte Destination, deaktiviert, um einen möglichst hohen Load auf den nahen Verbindungen zum Zielknoten zu induzieren. Derweil wird das Netzwerk dennoch zusammenhängend gehalten, da sonst bereits vier Ausfälle genügen würden, um die Destination vom Rest des Netzwerkes, zu trennen.

Zuletzt ist noch auf Traffic Pattern einzugehen. Im Kontext der Arbeit referenziert der Begriff des Traffic Patterns die Muster oder die Struktur des Datenverkehrs innerhalb des Netzwerkes. Auch diese sind in sich genommen facettenreich und anwendungsabhängig, dennoch lässt sich die Modellierung kondensieren[10, 3]. Die nachfolgend beschriebenen Pattern sind eher primitiver Ausprägung, die der Auswertung zweckdienlich sind und als Abwandlungen in jener verwendet werden. Zur Konkretisierung sei auch auf den anknüpfenden Abschnitt 4.3 und das Kapitel 5 verwiesen.

- **All to single Destination** beschreibt ein Traffic Pattern, in dem der gesamte Datenverkehr von allen anderen Knoten zu einem Zielknoten fließt. Dies stellt in Verbindung mit der Rückrichtung eine Analogie zu einem Client Server Modell[62] dar.
- **All to all** kennzeichnet ein Traffic Pattern, indem der Datenfluss zwischen allen Server-Knoten gleichverteilt zufällig zugeordnet wird. Respektiv wird aus der Produktmenge der Hosts unter Ausschluss aller reflexiven Paare eine Anzahl von Tupel gezogen, die Quellen und Ziele des Datenflusses modellieren. Gleichartig ist hier eine Analogie zu einem Netzwerk denkbar, indem jeder Host mit einem anderen chaotisch kommuniziert.

4.3. Methodik: Erfassung von Messwerten

Im nachfolgenden Kapitel 5 werden die Algorithmen bezüglich ihres Verhaltens unter Anwendung der Ausfallmuster verglichen, daher werden die Versuchsanordnung sowie die zu erhebenden Metriken und die zu erwartenden Schwierigkeiten und Einschränkungen nachfolgend erörtert. Zuallererst ist eine kurze Diskussion bezüglich der Auswahl aus verfügbaren Metriken geboten. In der Gesamtheit stehen verschiedene Metriken zur Verfügung, die jeweils unterschiedliche Aspekte der Netzwerkfunktionalität, Performance, der zugrunde liegenden Infrastruktur und der Qualität des zu betreibenden Services beleuchten. Folgend Tanenbaum et al.[62] und Bayilmis et al.[6] ergeben sich die grundlegenden Metriken einer Verbindung aus Latenz, Durchsatz, Paketverlusten und Jitter. Übergeordnet kann die Verfügbarkeit, Fehler-toleranz, Skalierbarkeit, Utilization oder QoS betrachtet werden. Abseits der Deliberation sind Metriken der Energieausnutzung oder andere erweiterte Ansätze. Obgleich Paketverluste (durch Fehler des Übertragungsmedium) und Jitter mit NetEm simuliert werden können, wäre zu erwarten, dass diese nur mittelbar die Varianz erhöht und keinen Beitrag zu Evaluation eines Fast Failover Verfahrens leistet. Des Weiteren ist die Skalierbarkeit aufgrund begrenzten Ressourcen nicht im Rahmen dieser Arbeit zu examinieren. Ähnlich würde QoS einen deutlich komplexeren Versuchsaufbau erfordern, da aktiv die Qualität des Datentraffic zu erfassen wäre.

4. Methodik und Implementation

Zur Versuchsanordnung: Das Set-up fokussiert die Applikation der Ausfallmuster an einem Torus-Netzwerk zur Evaluation der verschiedenen (Failover-)Algorithmen. Topologisch wird mittels der Softwareswitches von OpenVSwitch ein Torus konstruiert und an jeden der Switches ein Mininet-Host angehängt. Zum Erzeugen sinnvoller Ergebnisse wird jedes Ausfallmuster als Messpunkt der Ausfallrate mehrfach unabhängig unter den Routing-Algorithmen ausgeführt und dann entsprechend der Metrik aggregiert dargestellt. Folgerichtig ist zur Verringerung der Varianz eine möglichst hohe Repetitionsrate erstrebenswert. Allerdings ergeben sich aus dem beschränkten Zeitrahmen der Arbeit und der relativ langwierigen Erhebung zeitliche Grenzen. Zu erwähnen ist, dass zwecks Vergleichbarkeit innerhalb verschiedener Wiederholungen oder Ausfallmuster die Parameter eines Experimentes zu allen Iterationen zum Anfang gleich festgesetzt werden. Der Versuchsaufbau variiert daher in den Dimensionen der Topologie, um in Kombination mit dem Ausfallmuster möglichst aussagekräftige Ergebnisse innerhalb eines vertretbaren Aufwands zu ermöglichen. Hierneben werden als Link-Parameter zwischen den Switches eine Latenz von 10ms mit einer Bandbreite von 100 Mbit ohne Jitter appliziert. Zusätzlich sind die Links von den Switches zu den Hosts unbegrenzt, um keinen Einfluss auf die Bewertung des Failover-Algorithmus zu nehmen. Die Wahl dieser Parameter ist nicht arbiträr, sondern die Verzögerung einer Kante ist aus der Varianz und Timer Graduality des Linux Kernels begründet[27], auch wenn in einem Datacenter eher Latenzen innerhalb von Größenordnung schneller als eine Millisekunde anzutreffen sind[54]. Betreffend der Bandbreite scheint der gewählte Wert keine Überlastung zu erzeugen.

4.3.1. Erreichbarkeit und Latenz

Erreichbarkeit unter Ausfällen und damit implizit die Resilienz und sekundär eine gute Latenz stellen die fundamentalen Ziele eines Fast Failover Algorithmus dar[17, 11]. Ferner lässt sich durch die bekannten Verzögerungen der Kanten des Netzwerkes die grobe Weglänge eines Datenpaketes kalkulieren. Zur Erhebung der Erreichbarkeit wird das Programm Fping[30] verwendet, welches disruegt zu seinem Namensvetter Ping[31] gleichzeitig die Spezifizierung mehrerer Hosts erlaubt. Bei beiden Programmen handelt es sich um Diagnose-Tools, die mittels ICMP[1] Echo Request eine Rückmeldung der zu überprüfenden Hosts in Form eines ICMP Echo Replies anfordern. Anschließend wird anhand des Zeitpunktes des Eintreffens der Rückmeldung des anderen Hosts, Erreichbarkeit und Latenz des Round Trips eruiert.

Methodisch wäre auch eine Studie mittels TCP/UDP Ping[58] denkbar. Aber im analysierten Vergleich zwischen ICMP und TCP Pings attestieren Li et al. [40] dem TCP Ping eine bessere Zuverlässigkeit. Zu resümieren ist jedoch, dass dies auch vor allem auf äußere Einflüsse wie blockierende Firewalls zurückzuführen ist. Insofern besticht beim ICMP Ping die Simplität durch den Verzicht einer Serverkomponente und die ggf. geringe Paketgröße.

Fortführend werden zu jedem Tupel aus der Menge aller geordneten Paare von der Knotenmenge unter Ausschluss aller reflexiven Paare und symmetrischen Dopplung drei Messwerte erhoben. Als Latenz zu einem Tupel wird anschließend der Median aller Latenzen aus den zugestellten Paketen gewählt. Ein Tupel wird als erreichbar klassifiziert, sofern mindestens ein Ping erfolgreich war. Durch vorlaufende Experimente konnte das Verwerfen des ersten Pings als überflüssig betrachtet werden, da die Abweichung nachfolgender Messerwerte nicht signifikant war, vermutlich sowohl begründet an den statisch gefüllten ARP-Tabellen als auch an den statisch eingerichteten Routen in der Flow-Tabellen. Zur Analyse der Erreichbarkeit werden Paare als unerreichbar klassifiziert, sofern kein Ping zugestellt worden ist. Zwecks Studie der Latenzen werden nicht zugestellte Pings aus dem Datensatz entfernt, da eine andersartige Be-

handlung mittels Substitution durch Aggregation oder modellbasierte Vorhersage[41] nicht zielführend durch den eingebrachten Bias erscheint.

4.3.2. Durchsatz

Durchsatz, auch als »Throughput« bezeichnet, beschreibt die Menge an Daten, die über eine bestimmte Zeitspanne übertragen werden können[62]. Vornehmlich ist ein möglichst hoher Durchsatz zunächst ein grundlegender Indikator für eine opportune Ausnutzung verfügbarer Ressourcen und damit sekundär auch für den Erfolg des Routing Algorithmus. Allerdings ist eine feine Differenzierung unter Zuhilfenahme zusätzlicher Informationen wie des zugrundeliegenden Traffic Patterns, der Kennzahlen des Netzwerkes und der Topologie vonnöten, um nicht Trugschlüssen zwischen Ursache und Wirkung und damit zwischen erwarteter Performance und realer zu unterliegen. Beispielgebend sei ein Szenarium, in dem Datenverkehr zwischen zwei einzelnen beliebigen Knoten in einem Netzwerk vermeintlich maximalen Durchsatz erreicht, dabei aber hohe Belastung auf umliegenden Verbindungen durch Schleifen induziert, dies jedoch nicht an den beiden Messstellen der Knoten ersichtlich ist. Um diesem Umstand entgegenzuwirken, bietet sich die Auswertung erweiterter Traffic Pattern mit Beteiligung mehrerer Knoten und damit die Erhöhung der Anzahl der Messstellen an. Möglich ist auch die Inspektion angehängter Package Counter[50] der Links von OpenVSwitch als passiven Ansatz. Ersteres ist der verfolgte Ansatz in dieser Arbeit, wobei anzumerken ist, dass beides nicht exklusiv sein muss, letzterer Ansatz aber die Komplexität der Auswertung erheblich steigern würde.

In Einzelheiten erfolgt die Untersuchung des Durchsatzes mit dem Tool Iperf2[65] auf den beiden vorausgehend beschriebenen Traffic Pattern. Weiter handelt es sich bei Iperf2 um eine Software zur Ermittlung des Datendurchsatzes zwischen zwei Endpunkten mittels TCP oder UDP Stream. Die Software teilt sich dabei auf eine Server- und Clientkomponente auf. Die erzielte Datenrate ist je nach Modus an beiden Endpunkten abzulesen. Fortfahrend werden zu einer Messung alle erhobenen Datenraten je nach Traffic Pattern kumuliert und auf die verfügbare Bandbreite der Hosts normiert. In der Ausgestaltung wird der Datenverkehr beim All-to-All Pattern abweichend als beidseitiger (voll duplex) Fluss modelliert. Beim All-to-Single-Destination Pattern hingegen wird nur ein Simplex Fluss realisiert, da Iperf2 sonst überfordert ist.

Nachfolgend wird die Parametrisierung von Iperf2 bezüglich (TCP) Windows-Size und Paketgröße knapp erörtert. Zuerst ist laut Tanenbaum et al.[62] nur ein allgemeiner Zusammenhang zwischen Durchsatz und der TCP-Window-Size (Empfangsbuffer und Sendebuffer) zu erwarten, sofern die Buffergröße zu klein im Verhältnis zur Latenz, der Rundlaufzeit der Pakete und Bestätigungen (ACK) gewählt wird. Außerhalb der Betrachtung ist hier allerdings die Congestion-Control des Senders. Insgesamt ist sogar von einem noch geringeren Einfluss aufgrund von TCP Window Scaling (dynamische Anpassung der Größe des Fensters) auszugehen, sodass keine manuelle Anpassung der Window Size vorgenommen wird. Anschließend ist die Wahl der Paketgröße zu erläutern. Da Iperf2 nur die Rate der transportierten Daten ohne Packet-Header angibt[32], erscheint es zwecks Vergleichbarkeit sachdienlich, die maximale zulässige Größe eines Ethernet Frames zu wählen (1500 Bytes – Header). Im Rahmen wäre allerdings kein signifikanter Einfluss auf die Evaluation zu erwarten, wenngleich berichtete Datenraten mit Jumbo Frames linear höher wären (Datenanteil eines Netzwerkpaketes $\approx 95\%$ zu $\approx 99\%$ [62]).

Als negatives Fallbeispiel sei trotzdem die Publikation von Hardin et al.[24] angeführt, in der die Au-

4. Methodik und Implementation

toren zur Messung in Mininet mit Iperf2 bei spezifizierter Begrenzung der Rate der Kanten in Abhängigkeit der TCP Windows Size kurzzeitig vielfach höhere Werte als möglich sein sollten mit weiteren Irregularitäten beobachten konnten. Es war den Autoren nicht möglich, klare Ursachen zu identifizieren. Im Rahmen dieser Arbeit ließ sich das Verhalten im kurzen preliminary Testing auch nicht replizieren, außerhalb dessen, dass die berichtete Datenrate des Empfängers geringer war als die des Senders und in kurzen Zeitabschnitten die berichtete Datenrate die konfigurierten Raten in (deutlich) geringerem Umfang überschreitet kann. Letztlich entspricht aber die Datensumme und gemittelte Datenrate den Erwartungen. Die bidirektionalen Datenraten waren im Vergleich zur direktionalen bei Nutzungsgrad der verfügbaren Bandbreite etwas geringer, mit einem Verhältnis von 90% zu 70%, aber dennoch in sich konsistent. Dazu sei auch Anhang A.2 referenziert. Insgesamt können Mininet und NetEm aber als zuverlässig eingestuft werden[74, 23, 47, 33]. Zusätzlich sei für eine differenzierte Betrachtung speziell zu Mininet und NetEm auf den nachfolgenden Abschnitt verwiesen.

4.3.3. Einschränkung und Erwartung

Die Qualität der erhobenen Daten hängt artbedingt maßgeblich von der akkuraten Verzögerung und Ratenlimitierung von NetEm sowie Mininet ab. Daher ist eine kurze Diskussion bezüglich der Ressourcenzuteilung in Mininet und NetEm im Rahmen dieser Arbeit notwendig. Beginnend mit NetEm ist zunächst festzustellen, dass nach der empirischen Studie von Jurgelionis et al.[33] und der Forschung von Becker et al.[7] eine genaue Emulation von Verzögerung und Bandbreite nur möglich ist, sofern die Verzögerungszeiten nicht zu gering gewählt werden und die Timer-Interrupt-Frequenz (im Verhältnis) ausreichend hoch ist. Weiterhin ist mit Skalierungsproblemen zu rechnen, die sich jedoch außerhalb des Umfangs dieser Arbeit bewegen. Ferner ist in der Betrachtung von Mininet (in Verknüpfung mit OpenVSwitch) ebenfalls kein Hindernis in der Genauigkeit zu verorten, soweit keine realitätsfernen Anforderungen abseits der vorhandenen Ressourcen gestellt werden [74, 47, 16].

Ein weiterer Grund ist die unmittelbare Ressourcenallokation der laufenden Prozesse auf den einzelnen Hosts. Auch hier ist eine rechtzeitige Ausführung unabdingbar, da sonst Messfehler durch nicht versendete Pakete oder Ähnliches entstehen könnten. Der Problematik bewusst, gestattet Mininet (eigentlich) eine präzisere Ressourcenkontingentierung mittels Cgroups, gerade um der benannten Scheduler Starvation zu entgehen[64]. Zum Zeitpunkt dieser Arbeit war es allerdings nicht möglich auf diese Funktionalität zurückzugreifen, begründet wohl in einer inkompatiblen Kernel- und cgroup Version. Insgesamt scheint das Risiko nach der Forschung von Salah et al.[57] aber dennoch überschaubar zu sein, wenngleich nicht ausgeschlossen.

Zuletzt werden noch knapp die Erwartungen an die Ergebnisse konkretisiert, wenngleich ein Großteil der Datenanalyse explorativ erfolgt. Es ist offen zu erwarten, dass sich die beweisbaren Eigenschaften der Algorithmen innerhalb des Datensatzes bestätigen. Weiterhin kann spekuliert werden, dass der Low Stretch Algorithmus artbedingt durch seine Konstruktion einen besseren Durchsatz erzielt als Hamilton-basierte Verfahren, die das Paket einmal über alle Knoten leiten. Gleichmaßen ist mit (deutlich) besserer Erreichbarkeit und Latenz zu rechnen, gerade da unter Ausfällen eine geringere Fläche (Ausdehnung der Route über die Topologie) weniger Konfliktherde provoziert.

4.4. Architektur der Implementierung

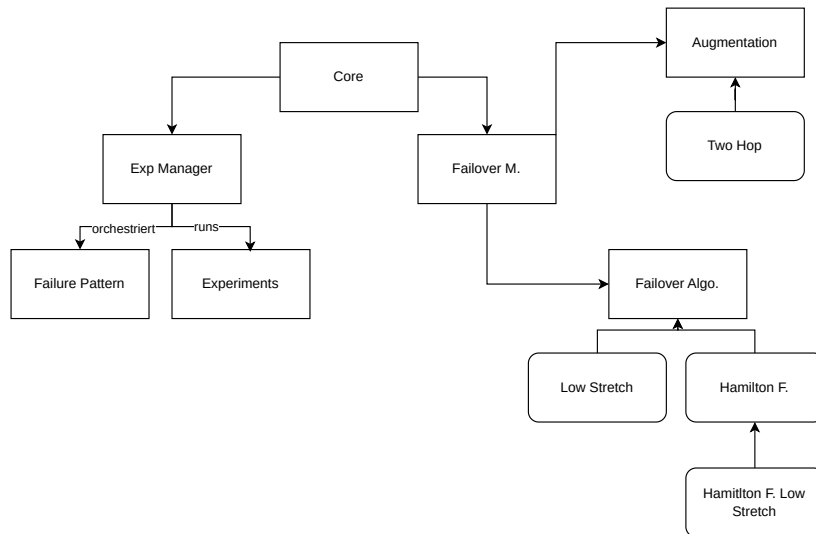


Abbildung 4.1.: Struktur der Implementierung

In diesem Abschnitt wird zunächst die grundlegende Architektur detailliert aufgegliedert. Im Anschluss erfolgt die individuelle Darstellung der Implementierung der Fast Failover-Algorithmen, genauer gesagt ihrer Modellierung in Openflow (Subsektion 4.4.1). Grundsätzlich teilt sich die Struktur der Implementierung wie in Abbildung 4.1 in drei Teile ein: den Core, den Experiment Manager und den Failover Manager. Anschaulich orchestriert der Core das Zusammenspiel beider Manager und initialisiert am Anfang das Mininet Netzwerk anhand der übergebenen Parameter. Der Failover Manager administriert anschließend die Injektion der Openflow Regeln des Fast Failover Routing Algorithmus und Augmentationen in das Mininet Netzwerk mittels `ovs-ofctl` [70] als Systemaufruf. Bei `ovs-ofctl` handelt es sich um ein Administrationstool von OpenVSwitch zur Manipulation von Konfigurationen und Zuständen eines Switches. Schließlich ist der Experiment Manager für die Deaktivierung der Kanten nach einem ausgewählten Ausfallmuster und die Abwicklung der Experimente sowie das Speichern der erzeugten Daten zuständig.

Die Umsetzung der tatsächlichen Ausfallmuster aus Abschnitt 4.2 erfolgt durch Iteratoren, die die zu deaktivierenden Kanten je Schritt zurückgeben. Der Experimente Manager lässt anschließend die assoziierten Kanten ausfallen und führt die Liste der Experimente seriell aus. Die Resultate werden dabei in ein JSON File eingetragen. Zusätzlich ist eine aggregierte Exekution mehrerer Ausfallmuster zu einer Iteration sequenziell möglich, wobei die Kanten beim Wechsel zwischen den einzelnen Ausflussmustern wiederhergestellt werden.

Alle Experimente sind allgemein von einer abstrakten Experimentklasse abgeleitet, die ein grundlegendes Interface zur Ausführung bereitstellt. Anknüpfend folgt daher eine grobe Auflistung und Charakterisierung sämtlicher Experimente:

1. **FpingMultiBatch:** Eröffnend werden bei diesem Experiment mehrere Pings zwischen allen Hosts zur Erreichbarkeits- und Latenzmessung ausgeführt. Insgesamt beinhaltet dies die Ausführung von

4. Methodik und Implementation

Fping und anschließend das Parsen der Ausgabe mit Zuordnung der Messwerte nach Produktmenge der Knoten. Weiterhin lässt sich die Anzahl der Pings pro Host festsetzen. Es besteht dabei die Möglichkeit, Fping auf mehreren Hosts parallel als Batch auszuführen, um die Dauer der Ausführung erheblich im Vergleich zu sequenziellen einzelnen Ping-Programmaufrufen zu beschleunigen, gerade auch weil Fping zusätzlich die Spezifizierung mehrerer Hosts direkt erlaubt[30]. Dabei konnte keine nennenswerte Abweichung zum singulären Fping Experiment (Batchgröße von Eins) festgestellt werden, sofern die Batchgröße nicht unangemessen groß gewählt wurde.

2. **IperfAllToAll:** Als Nächstes wird bei dem Experiment das Traffic Pattern »all to all« realisiert, indem korrespondieren Iperf2 in Client Konfiguration als Quelle und als Ziel in Server Konfiguration zwischen den gezogenen Paaren im voll duplex TCP Modus betrieben wird. Angegliedert ist nach Ausführung eine Auswertung der Ausgaben von Iperf2, die die Übertragungsrate je Client und Server zusammenstellt. Zusätzlich besteht Möglichkeit, die Parameter von Iperf2 direkt zu manipulieren und so musterhaft ein UDP basierten Test durchzuführen.
3. **IperfSingelDest:** Schließlich verwirklicht das Experiment das verbleibende Traffic Pattern »All to singel dest«. Gleichartig zum Traffic Pattern wird der Host der erwählten Destination im Servermodus betrieben und die verbleibenden Hosts im Clientmodus. Zur Bandbreitenmessung wird wieder Iperf2 verwendet. Zur Optimierung der realistischen Genauigkeit kann zusätzlich die isochrone Betriebsart gewählt werden, die den Datenverkehr mit der Frequenz von Frames pro Sekunde und Load definiert und durch Mittelwert und Standardabweichung modelliert ist[32]. Dies ist einer Videoübertragung nachempfunden. Hierneben erfolgt wieder die Erhebung der Datenrate der Clients und des Servers anhand Ausgaben von Iperf2. Eine Analyse des Frame Timings im isochronen Betriebsmodus war hingegen nicht zuverlässig möglich.

4.4.1. Implementierung der Fastfailover Algorithmen und Augmentation

Die Realisierung der Algorithmen ist unbedingt nicht gradlinig, denn Openflow erhebt (wie erwähnt in Subsektion 2.1.2) konstruktionsbedingt erhebliche Beschränkungen an eine Implementierung durch die feste Spezifikation an Routingverhalten in einem Switch[50], gerade da konzeptuell im Fast Failover Routing kein Controller beteiligt ist. Zweckmäßig wird daher nachstehend ein Schema ausgearbeitet unter Erläuterung der Partizipation und Aufgaben des Fast Failover Manager. Lancierend sind aber die konkreten Hindernisse festzuhalten. Einleitend kann das Matching in eine Flowentry nur als Konjugation (»AND« Verknüpfung) der Match Felder eines Paketes erfolgen. Ebenso kann eine ausgefallene Kante nicht als Match Feld definiert werden, sodass jene Bedingung ausschließlich über die Verwendung von Fast Failover Gruppen umgesetzt werden können. Weiter ist zeitliche Abfolge zu beachten, sodass das Matching nicht gleichzeitig erfolgt, sondern vor/nachgelagert der Anwendung der Fast Failovergruppe innerhalb der ingress und/oder egress Flowtabel. Die Fast Failover Gruppe hingegen kann verglichen werden mit einer impliziten Disjunktion, die die erste aktive Kante innerhalb der Reihenfolge bespielt. Sollte es zu keinem Match in einem Flowtabel kommen, so wird Paket standardmäßig gedroppt.

Resultierend lässt sich ein (vereinfachtes) grundlegendes Schema entwerfen: Vorrangig muss eine einzelne Routing Bedingung des Algorithmus in disjunktive Normalform umgesetzt werden, wobei die Disjunktionen über die nachrangige Anwendung von Prioritäten und Flowtable misses vollzogen werden können. Ähnlich sind Negation zu behandeln, nur dass alle Fälle konträr der Negation als Flow Entry

4.4. Architektur der Implementierung

zu spezifizieren sind. Im Anschluss an das Matching hat die Überleitung an eine geeignete Fast Failovergruppe innerhalb der Action zu erfolgen. Abschweifend von dem Schema können Verschaltungen aber durchaus mit mehreren sequenziellen Flowtable umgesetzt werden oder es sind sogar differenzierte Ansätze über das sequenzielle Modifizieren und Matchen der Pakete denkbar.

Schließlich bleibt es noch, die Beteiligung des Fast Failover Manager zu erläutern. Zunächst sind wieder alle Fast Failover Algorithmen wie die Experimente aus einer eigenen abstrakten Klasse abgeleitet. Neben der Exekution der Algorithmen obliegt dem Manager zusätzlich, die Port Nummer der Switches zuzuordnen und eine geeignete Fast Failover Gruppen zu den angeforderten Kanten zu lokalisieren. In Einzelheiten wird zum Start in jedem Switch über die Permutation aller Kanten zu angrenzenden Switches eine Menge von Gruppe erstellt, wobei die »beobachtete« Kante (watch port) immer zur gleichen Ausgabe (bucket) korrespondiert. Schlüssig lassen sich daher nach aktiven Kanten alle Priorisierungen der Ausgabe realisieren.

Schließlich sei noch das Konzept der Augmentation beleuchtet. Hierbei handelt es sich auch um Openflow Implementierungen, die konsekutiv priorisiert nach dem Algorithmus angewandt werden können, um das Verhalten in jeglicher Hinsicht zu modifizieren. Dazu werden die Einträge der Augmentation explizit höher priorisiert als die des Algorithmus und finden daher zuerst Anwendung.

4.4.2. Implementierung: Hamilton Zykale Fast Failover

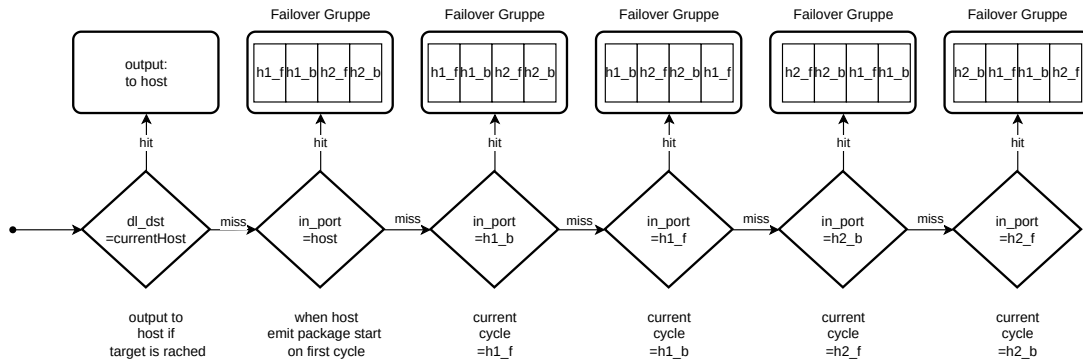


Abbildung 4.2.: Flussdiagramm Openflow Hamilton Fast Failover

Nun zur konkreten Implementierung des Hamilton Zykle Fast Failover auf Chiesa et al.[11]. Zugleich sei für die explizite Konstruktion der Hamilton Zyklen für den Torus auf Abschnitt 3.1.1 verwiesen. Abstrahierend dargestellt wird zu Beginn mit der ersten Regel sichergestellt, dass das Datenpaket, emittiert von einem Host, auf den nächsten aktiven Hamilton Zyklus ausgebracht wird. Jene sind in Abbildung 3.1 als $h_{1f} - h_{2b}$ deklariert, wobei die Ziffer den Zyklus und der Charakter die Laufrichtung mit **forward** und **backwards** artikuliert. Weiter sollte ein Paket den Switch verbunden an den Zielhost erreichen, so wird höchst priorisiert das Paket über die direkte Verbindung an jenen Zielhost geroutet. Die verbleibenden Regeln entsprechen hingegen alle dem gleichen Muster. Die Rangfolge im Flowtable entspricht aber der numerischen Ordnung der Hamilton Zyklen. So wird jeweils immer am eingehenden Port der aktuelle Hamilton Zyklus identifiziert (dieser entspricht als Eingangsport gegensätzlich der Laufrichtung). Anschließend erfolgt die Ausgabe an eine Failover Gruppe, die auf den ersten aktiven Zyklus beginnend

4. Methodik und Implementation

mit dem aktuellen Zyklus das Paket emittiert. Dabei wird abweichend von Chiesa et al.[11] die Sequenz der Hamilton Zyklen nicht nach Erreichen des letzten Zyklus beendet, sondern ggf. zyklisch rotiert, sodass ein Paket laufend auf dem letzten Zyklus in Rückrichtung wieder auf den ersten geroutet werden kann, sofern eine Kante des Zyklus ausfällt. Stringent können so zwar (mehr) Schleifen induziert werden, allerdings können potenziell mehr Ausfälle toleriert werden und es bieten sich Synergieeffekte mit einer alternativen anderen Art der Konstruktion der Ortung der Hamilton Zyklen an. Dies wird folgend nun erläutert.

Ein Hamilton Zyklus umfasst alle Knoten der Topologie[69]. Korrespondierend kann daher im ungünstigsten Fall eine Wegstrecke über alle Knoten zurückgelegt werden, obwohl die Knoten direkt benachbart sind[11, 20]. In Intention der Optimierung wird daher initial der Zyklus mit der geringsten Distanz zum Ziel erwählt, bei Beibehaltung des restlichen Regelschemas. In der Ausgestaltung wird nun die erste Regel derart erweitert, dass der erste Switch anhand der Destination direkt den Zyklus adressiert, der die kleinste Distanz verspricht bzw. die korrespondierende Failover Gruppe. Somit erfordert dies für die Destination eine derartige Regel in jedem Switch. Unweigerlich garantiert der Ansatz keine bessere Wegstrecke bei inopportun ausgefallenen Kanten, weil so die gleiche Situation wie im vorherigen Algorithmus konstruiert werden kann. Zusätzlich werden mehr Einträge benötigt. Anderenfalls würde eine allgemeine Anpassung der Rangfolge der Hamilton Zyklen quadratisch mehr Regeln induzieren (ein Switch müsste so jede individuelle Rangfolge anhand eines Quelle/Ziel Paares identifizieren), ohne vermeintlich bessere Erfolge abseits nah verorteter Ausfälle zu versprechen.

4.4.3. Implementierung: Low Stretch Fast Failover

Nachfolgend wird das Fast Failover Verfahren aus Foerster et al.[20] realisiert. Vorher sei dazu auf die Subsektion 3.1.2 zur Charakterisierung verwiesen. In analoger Vorgehensweise lässt sich anhand der Weiterleitungsregeln ein OpenFlow Schema entwickeln, das im Folgenden ausgearbeitet wird. Vorab ist zu beachten, dass die Regeln zu jedem Knoten, der als Ziel dienen soll, gesondert konstruiert werden müssen. Jene lassen sich anschließend anhand des Zielknotens identifizieren, genauer mit der dl_dst in OpenFlow. Die Regeln zur Weiterleitung an den direkten Host sind dieses Mal nicht aus Übersichtsgründen nicht eingezeichnet.

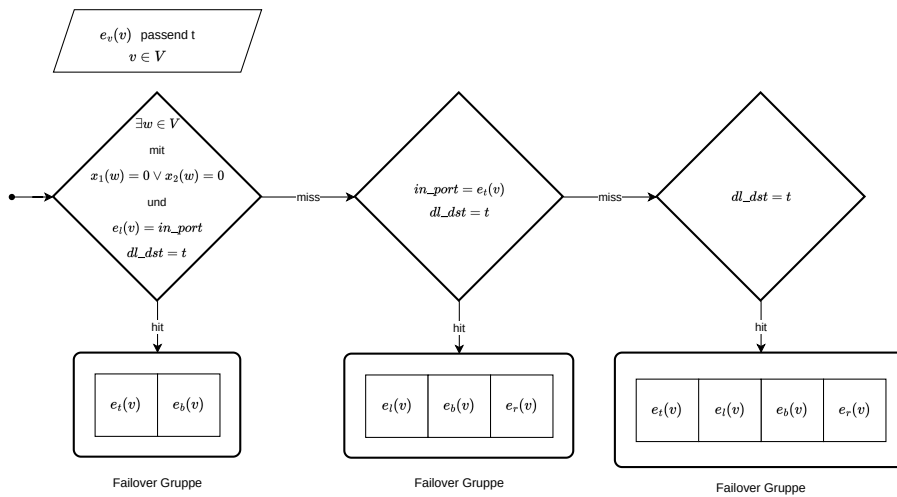


Abbildung 4.3.: Flussdiagramm Openflow Low Stretch Verfahren Fast Failover

Einleitend sei auf die modifizierte Rangfolge zu den konzeptuell korrespondierenden Bedingungen des (Ur-)Weiterleitungsschemas aufmerksam gemacht. Dies ist wieder aus der Architektur der Matching bedingt, insbesondere der Umsetzung der Regel I, dass ein Routing über e_t erfolgen muss, sofern die Kante selbst nicht ausgefallen ist oder die Eingangskante ist. Beginnend mit dem OpenFlow-Schema: Die erste Regel findet nur Anwendung, wenn es einen Knoten gibt, der auf einer der Achsen platziert ist und dessen linksseitige Verbindung (e_l) innerhalb des aktuellen Knotens endet. Die Failovergruppe emittiert abseits dessen trotzdem erst zu e_t , da nach (Ur-)Schema e_t zu verwenden ist, solange der eingehende Port nicht e_t ist (und die Verbindung aktiv). Konkludent ist das der Fall, da der in_port nach Bedingung e_l sein muss (Umsetzung Regel I und II). Anschließend wird bei in_port von e_t an eine Failovergruppe mit kongruenter Priorisierung des Ur-Schemas weitergeleitet (dies entspricht Regel I und III des Ur-Schemas). Schlussendlich wird in der dritten Regel analog verfahren, nur mit Bespielung von e_t als erste Priorisierung der Failovergruppe (auch Umsetzung Regel I und III des Ur-Schemas).

4. Methodik und Implementation

4.4.4. Implementation: Perfect Resiliency within 2-Hops

Nachfolgend wird die Umsetzung der Two-Hop Augmentation beschrieben, abgeleitet aus Foerster et al.[18]. Es sei auch auf Subsektion 3.1.3 verwiesen. Zuerst ist jedoch festzustellen, ob die Topologie eines Torus überhaupt für eine Anwendung geeignet ist, denn es kommen nur solche Sourceknoten infrage, bei denen selbst nach einem Ausfall einer oder mehrere Kanten ein Weg der Länge zwei zum Ziel bestehen kann. In Abbildung 4.4 sind die vier Knoten, die in einem Source to Target Model in Betracht kommen, grün markiert. Es sei darauf hingewiesen, dass dies für explizite Nachbarn der Destination nicht zutrifft, da innerhalb eines Ausfalls der adjazenten Kanten nur eine alternative Route über einen der grün markierten Knoten bestünde, jedoch mit einer Weglänge von drei. Nahezu analog ist zu den gelb markierten Knoten des Schaubildes zu argumentieren.

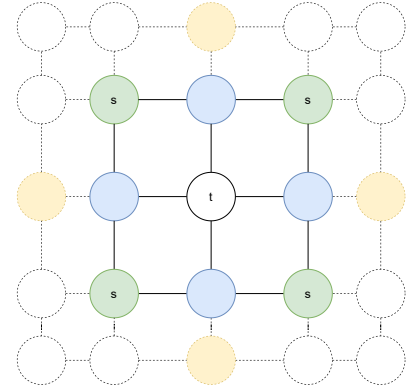


Abbildung 4.4.: Sourceknote (S) in Grün, wenn nach einem Ausfall noch Wege der Länge 2 zu t bestehen. Direkte Nachbarn von t in Blau und Knoten mit einem Weg zu Source in Distanz 2 in Gelb. Für ein Torus $\geq 5 \times 5$

Weiter zur Charakterisierung: Abstrakt sendet die Quelle das Paket zu einem geeigneten Nachbarn, der das Paket bei intakter Verbindung zum Ziel zustellt oder bei ausgefallener Kante das Paket zum Sender zurückschickt. Die Quelle ist nun in der Lage, über den Eingangsport die unmögliche Zustellung zuzuordnen und einen anderen qualifizierten Knoten zu probieren.

Nun zur Realisierung in Openflow. Um die Auswirkung auf nicht geeignete Pakete gering zu halten, wird auch Source Matching verwendet. Anders wäre vielleicht sonst auch eine Nutzung des Theorem 6.2 [18] denkbar. Weiterhin ist eine Differenzierung zwischen dem Regelstamm für Quelle und Übergangsknoten vonnöten. Zusätzlich wird das Schema wieder für alle Knoten angewendet und implizit auch mehrfach, da wie angeführt mehrere Knoten für ein Ziel geeignet sind.

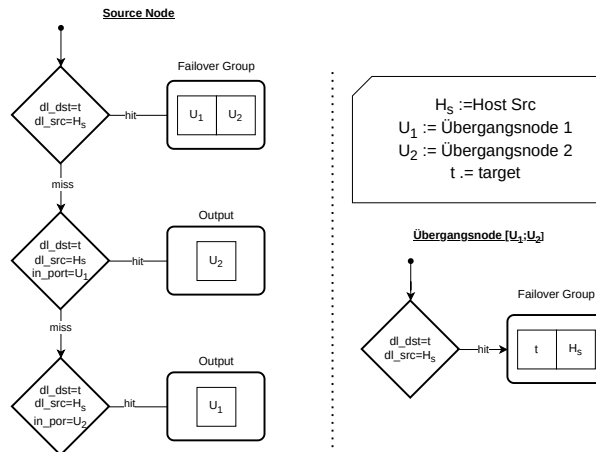


Abbildung 4.5.: Flussdiagramm Openflow Augmentation Two-Hop

4.4. Architektur der Implementierung

In Betrachtung des Source Knotens, genauer eigentlich des Switches, der an den emittierenden Host angehängt ist, wird jenes Paket anhand der Adresse des Hosts und des Ziels als ein qualifiziertes identifiziert und so an eine Failovergruppe mit den beiden Übergangsnodes weitergeleitet. So wird das Paket an einen (aktiven) Übergangsnode ausgegeben. Nahezu analog identifiziert der Übungsnode das Paket und entsendet es nach Zustand der adjazenten Kanten zum Ziel oder zurück (modelliert als Failover Gruppe). Im nachgeordneten Fall erhält der Sourceswitch das Paket und kann die Zustellung als fehlgeschlagen anhand des Eingangsportes von dem Übergangsnode erkennen. Somit wird der andere Übergangsknoten frequentiert. Hier ist die Vorgehensweise abschließend gleichartig, nur dass im negativen Fall keine weiteren Übergangsnodes verbleibt. Daher sei darauf hingewiesen, dass vermeintlich keine Verbesserung der Resilienz mit der Augmentation in den betrachteten Verfahren zu erzielen ist.

5. Evaluation

In diesem Kapitel werden die Ergebnisse der Evaluation gemäß der im vorigen Kapitel beschriebenen Methodik dargestellt. Dazu erfolgt eine Analyse mit angeschlossener grafischer Repräsentation der erhobenen Daten. Weiterhin werden die Metriken getrennt aufgeschlüsselt (Sektion 5.1 und 5.2) und die genauen Versuchsp Parameter innerhalb der Bildunterschrift aufgeführt. Die konkrete Einordnung und Diskussion der Ergebnisse erfolgt in der sich im nächsten Kapitel anschließenden Diskussion (Kapitel 6). Weiterführend wird am Ende der Evaluation die Argumentation bilanziert (Sektion 5.3).

5.1. Latenz und Erreichbarkeit

Innerhalb dieser Sektion werden die in den Subsektionen präsentierten grafischen Darstellung der Latenz und Erreichbarkeitsquote analysiert. Zuerst sei aber noch angemerkt, dass die Boxplots zu dem Durchsatz logarithmisch skaliert sind und zusätzlich nicht zugestellte Pakete nicht in der Statistik geführt werden (Lower is better). Hinsichtlich der Latenz werden nicht zugestellte Pakete in Anteil in Hundertstel zur entsprechenden Iteration des Pattern angegeben (Lower is better).

5.1.1. Failure pattern: Random Edges

Random Edges: In Studie der Erreichbarkeit (Abbildung 5.1) ist insbesondere bemerkenswert, dass alle Verfahren keine Verluste bis zu einer Ausfallrate von vier Kanten zeigen und der Low Stretch Algorithmus sogar bis nahezu acht Ausfälle toleriert, mit einer Abweichung einer Stichprobe. Beim Vergleich der beiden Hamilton basierten Verfahren zeigt sich eine annähernd gleichartige Entwicklung der Erreichbarkeitsquote, wobei das Low Stretch Hamilton Verfahren zu Beginn einen höheren Median, aber geringere Ausreißer nach oben aufweist. Zum Ende ist eine Umkehrung dieses Phänomens bezogen auf den Median zu taxieren. Ansonsten ist der Abstand der Maxima überschlägig konstant. In Einordnung unter der Aggregation der Rohdaten ist weiterhin für die Hamilton basierten Verfahren markant, dass zuerst nur ein geringer Teil der Stichproben bis zu fünf Kantenausfällen überhaupt eine Reduktion der Quote provoziert, der Anteil danach aber rapide ansteigt. Entlang bis zu neun Ausfällen kann aber immer noch eine Stichprobe erhoben werden, bei der alle Pakete zugestellt werden können. Dies ist bei dem Low Stretch Verfahren bis zu elf Kantenausfällen der Fall und auffallend auch abermals bei 15 bis 16 Kantenstörungen. Prägnant ist zudem die ab dem fünften Ausfall zunehmende Standardabweichung bei allen Verfahren, verwendbar als Indikator für die Spannweite zwischen den einzelnen Stichproben. Bei Hamiltonbasierenden Verfahren ist allerdings eine abflachende Degression nach neuntem Kantenausfall zu observieren.

In Betrachtung der Latenz (Abbildung 5.2) ergibt sich ein differenziertes Bild. Zuerst auffallend ist die um mehrere Größenordnungen geringere Latenz des Low Stretch Verfahrens. Bezeichnend ist zusätzlich der nahezu konstante Median über den vollständigen Verlauf, als Indikator eines (geringen) Einflusses mehrerer Ausfälle auf die gesamte Population der Latenzen. In der Betrachtung offenbart sich allerdings

5. Evaluation

eine Schwäche des Maßes, denn es kann ein stetiger Anstieg der Latenzen im oberen 25-Quantil sowie im Maximum nachgehalten werden. Implizit ist daher durchaus ein Einfluss auf einen Teil der Population anzunehmen. Innerhalb der Hamilton Verfahren ist zuerst festzustellen, dass das Low Stretch Hamilton Verfahren eine vollumfänglich geringere Latenz erzielt als das andere Hamilton Verfahren. In Betrachtung der Distribution scheint aber eine vergleichbare Standardabweichung erfasst zu werden. Es ist bei beiden Verfahren eine Degression der gesamten Population zum Ende zu beobachten, die allerdings vermutlich auf eine deutlich kleiner werdende Stichprobengrößen (deleted Values) zurückzuführen ist. Zuletzt sei noch darauf hingewiesen, dass beide Verfahren auch eine Progression des oberen 25-Quantil aufweisen, diese aber bei dem Low Stretch Hamilton Verfahren deutlicher ausgeprägter ist.

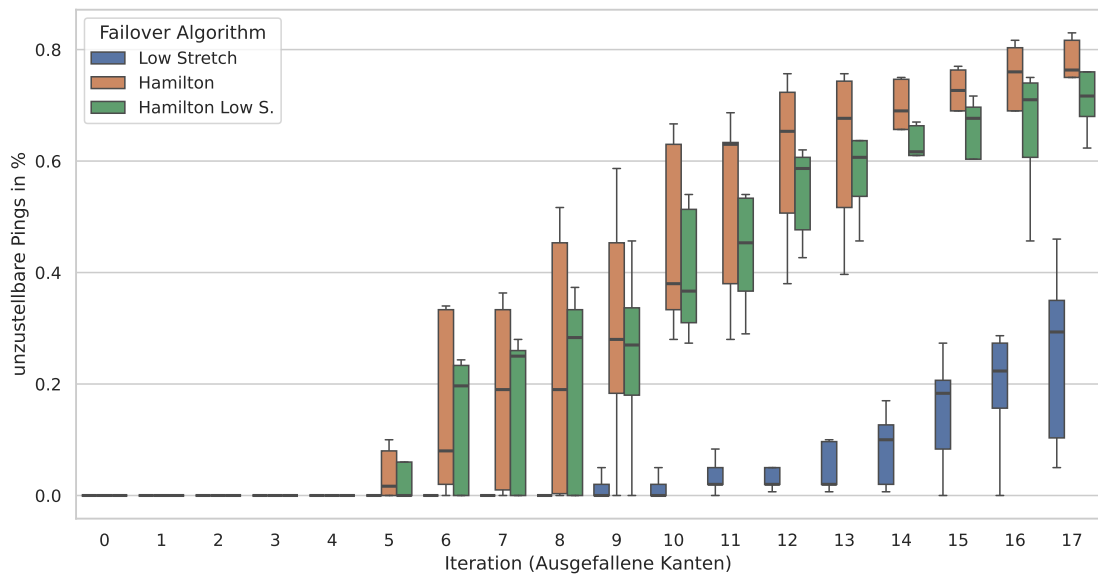


Abbildung 5.1.: Erreichbarkeit unter zufälligen Kantenausfällen. 5×5 Torus.

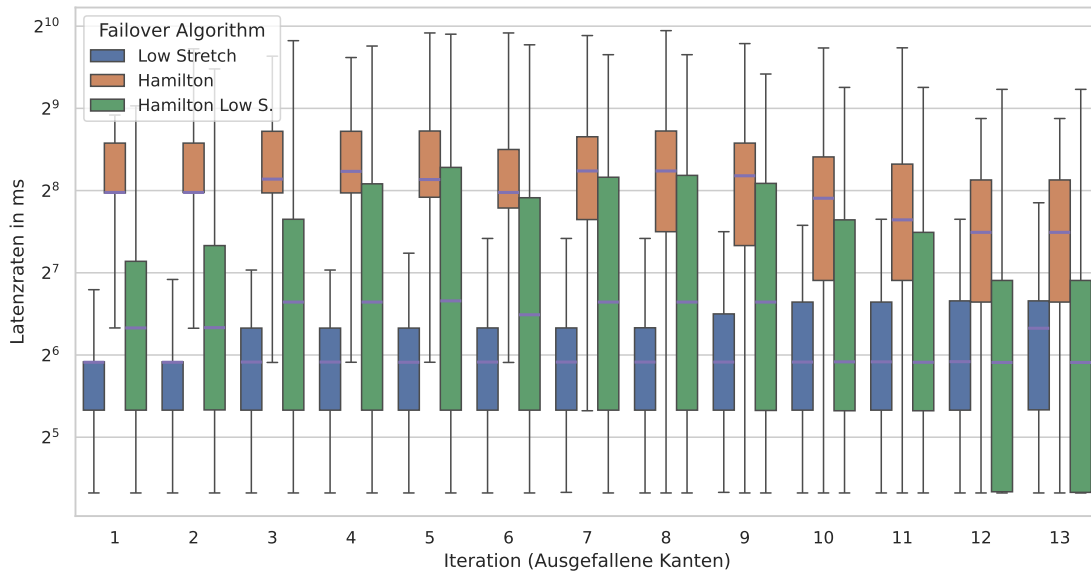


Abbildung 5.2.: Latenzen unter zufälligen Kantenausfällen. 5×5 Torus.

5.1.2. Failure pattern: Random Nodes

Random Nodes: In Analyse der Erreichbarkeit (Abbildung 5.3) setzt sich das bekannte Muster fort. Das Low Stretch Verfahren weist einen erheblichen (leicht progressiven) Abstand zu den Hamilton basierenden Verfahren in der Erreichbarkeitsquote auf. In der Einordnung der Hamilton Verfahren zeigt sich hingegen ein leicht degressiver Abstand mit vorteilhaften Quoten für das Low Stretch Hamilton Verfahren. Bemerkenswert ist zudem, dass alle Verfahren unter dem Ausfall eines Knotens doch noch annähernd 90% der Pakete zustellen, obwohl der ausgefallene Knoten bereits einen Anteil von 2% ausmacht. Entlang des Verlaufes ist weiter hervorzuheben, dass selbst unter einem Ausfall von zehn Knoten noch über 50% der Pakete vom Low Stretch Verfahren zugestellt werden können. Eklatant ist des Weiteren die minimale Standardabweichung des Low Stretch Verfahrens. Es scheint beinahe unbedeutend zu sein, welcher Knoten ausfällt. Im Kontrast dazu ist eine nicht unerhebliche Spannweite bei den Hamilton basierten Verfahren zu beobachten, die aber auch über den Verlauf abnimmt.

In der Auswertung der Latenz (Abbildung 5.2) zeigt sich ein analoges Bild zu den zufälligen Kantenausfällen. Prägnant sind die allgemein etwas höheren Latenzen, was vermeintlich in der größeren Topologiegröße und den damit einhergehenden längeren Wegen begründet ist. Beachtenswert ist zudem die degressive Entwicklung großer Teile der Population zum Ende bei den Hamilton basierten Verfahren. Dies ist aber wohl wieder den gelöschten Werten zuzusprechen. Ansonsten können hingegen bei dem Low Stretch Verfahren wieder ein progressiverer Median und ein progressives Maximum observiert werden.

5. Evaluation

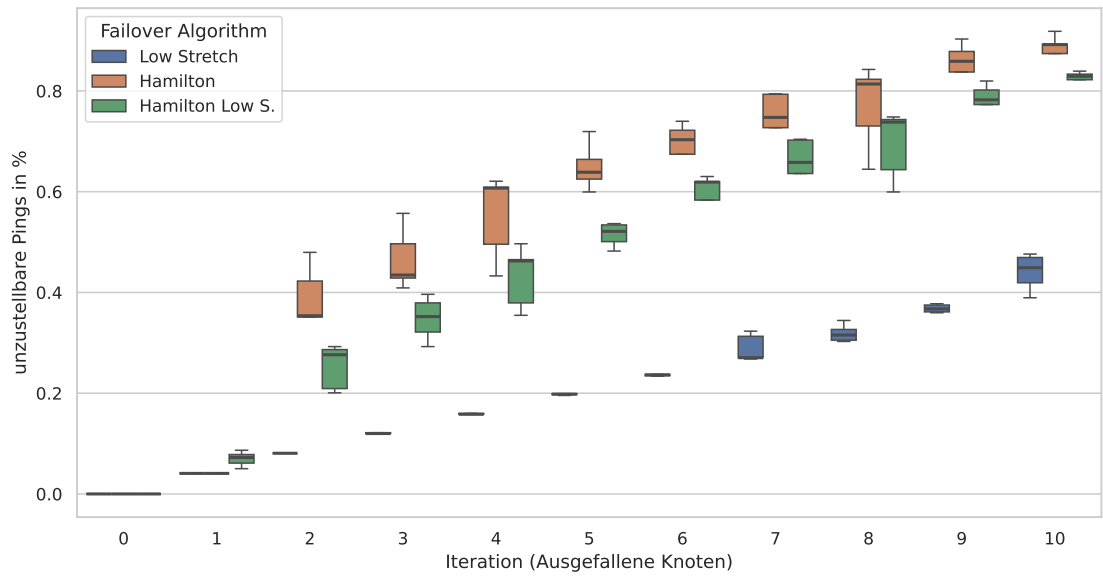


Abbildung 5.3.: Erreichbarkeit unter zufälligen Knotenausfällen. 7×7 Torus.

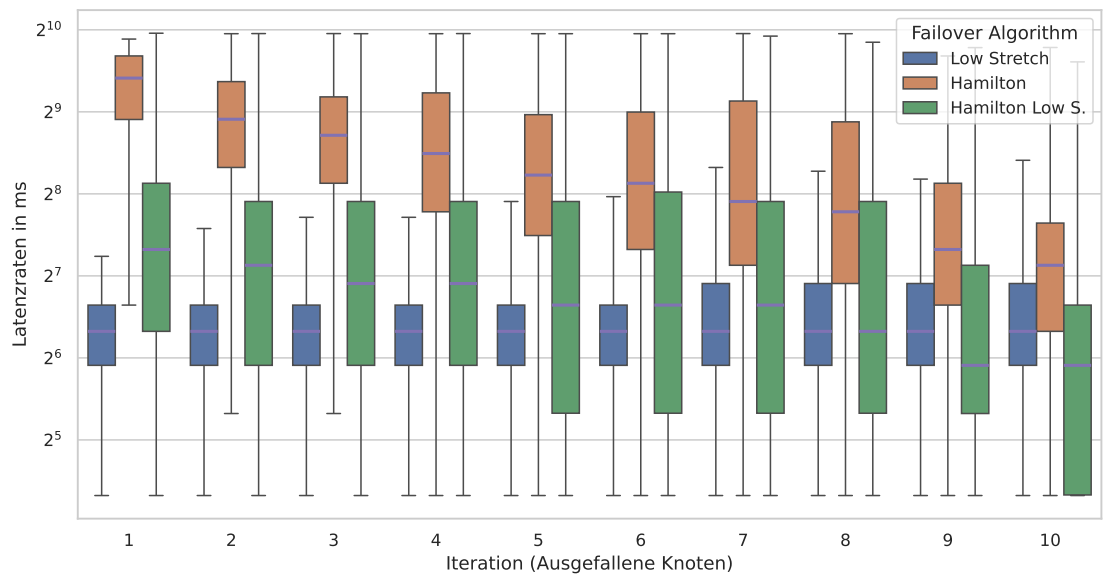


Abbildung 5.4.: Latenzen unter zufälligen Knotenausfällen. 7×7 Torus.

5.1.3. Failure pattern: Clustered Failiures

Clustred Failures: In der Untersuchung des Clustered Failiures Pattern ist es zuerst angebracht, die Geometrie zu illustrieren. Bei dem »Step«-Derivat ist ein Epizentrum mit immer größerer Ausdehnung je Iteration zu erwarten, im Gegensatz zu der »Node«-Variante mit mehreren kleinen Epizentren mit geringer Ausdehnung.

Zur Erhebung der Erreichbarkeit (Abbildung 5.5 und 5.7) unter der Betrachtung der Step Variante steigt mit zunehmender Ausdehnung des betroffenen Gebietes die Ausfallrate. Zur Einordnung ist je Iteration im Schnitt eine Zunahme von drei bis fünf Kanten zu erwarten, aber mit stark steigenden Tendenzen über den Verlauf. Bezeichnend ist hier aber wieder die deutlich günstigere Quote für das Low Stretch Verfahren, gefolgt von dem Hamilton Low Stretch Verfahren. Prägnant ist zudem die doch (verhältnismäßig) große Standardabweichung zwischen den Stichproben, insbesondere auch im Low Stretch Verfahren zwischen der vierten und zwölften Iteration. Dies ist aber wohl auch auf die unterschiedlichen Ausdehnungen der einzelnen Instanzen des Failure Patterns zurückzuführen. Auffallend außerdem zur ersten Stichprobe sind keine signifikanten Verluste außerhalb des Low Stretch Hamilton Verfahrens zu verzeichnen.

In Reflexion des Node-Derivats lassen sich starke Parallelen zu zufälligen Knotenausfällen erkennen, mit aber einer doch bedeutend größeren Standardabweichung. Die Parallelen sind mit einer räumlich meist relativ ähnlichen Ausdehnung eines Ausfalls zu begründen. Die Standardabweichung folgt wiederum analog der Argumentation bei den Failure Pattern Instanzen.

Bei Studie der Latenz (Abbildung 5.6 und 5.8) unter der Inspektion der Step Variante ist die Latenz des Low Stretch Verfahrens wieder um Größenordnungen besser als die der Hamilton Verfahren. Interessanterweise kann primär eine Degression des Medians und sekundär eines Großteils der Population innerhalb der Hamilton basierten Verfahren beobachtet werden. Dieses Phänomen tritt jedoch erst ab der 13. Iteration innerhalb des Low Stretch Verfahrens auf. Zu betonen ist die deutlich bessere Latenz des Low Stretch Hamilton Verfahrens im Vergleich zum normalen Hamilton Verfahren. Ansonsten ist eine degressive Standardabweichung bei den Hamilton Verfahren zu verzeichnen.

Zuletzt bleibt die Analyse des Node-Derivats: Es ist wieder die gleiche Abstufung in der Ordnung der Verfahren zu verzeichnen. Im Übrigen ergeben sich keine Auffälligkeiten abseits dessen, dass die Messdaten den Knotenausfällen ähneln.

5. Evaluation

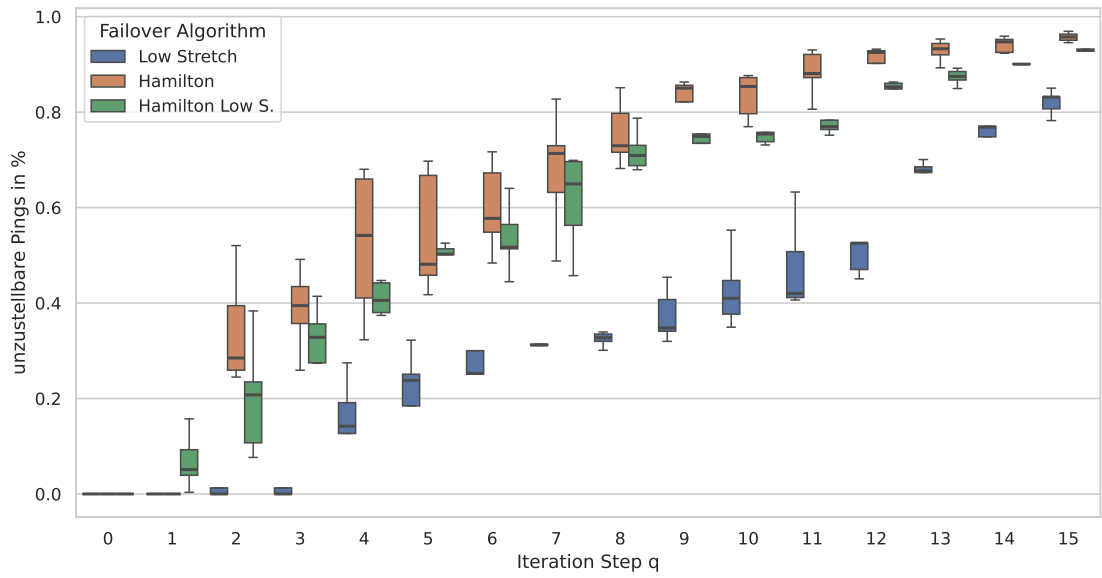


Abbildung 5.5.: Erreichbarkeit unter Clusterfailure mit $p = 0.9$ und $q \in (0.3, 0.9)$. 7×7 Torus.

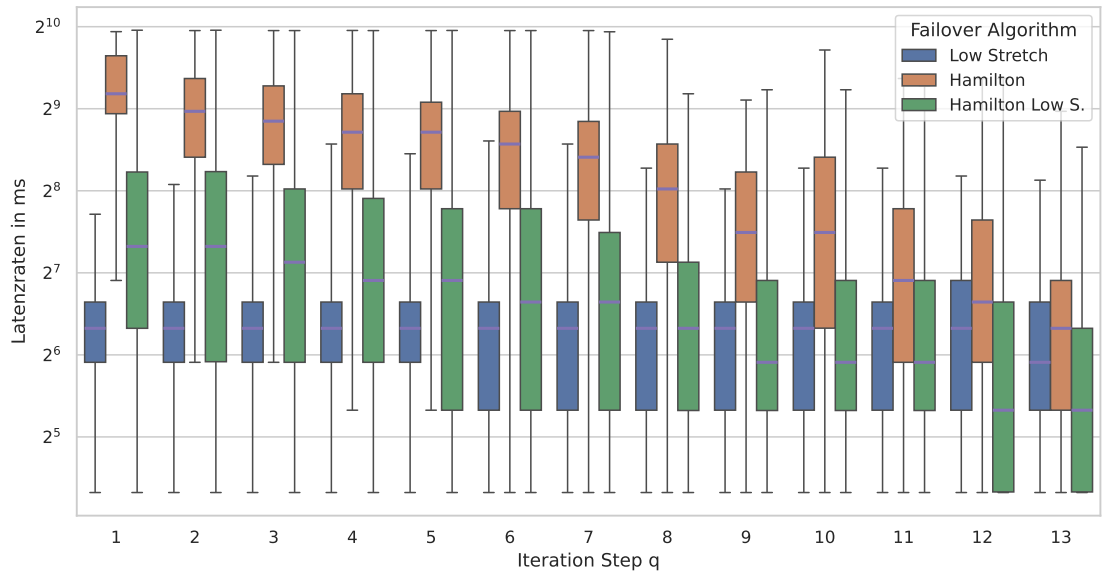


Abbildung 5.6.: Latenzen unter Clusterfailure mit $p = 0.9$ und $q \in (0.3, 0.9)$. 7×7 Torus.

5.1. Latenz und Erreichbarkeit

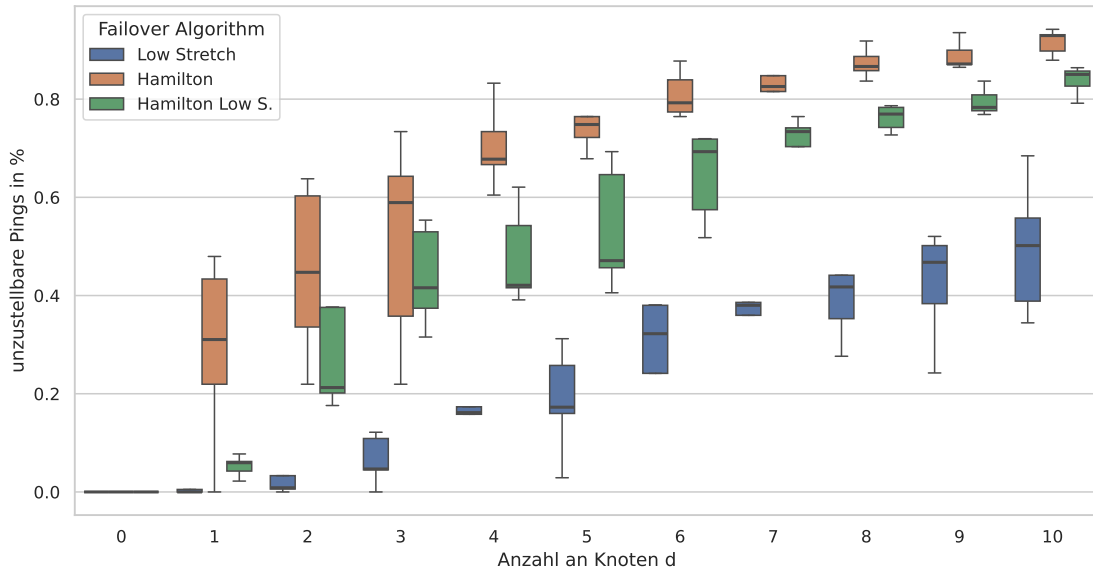


Abbildung 5.7.: Erreichbarkeit unter Clusterfailure Node. $p = 0.9 \wedge q = 0.34$. 7×7 Torus.

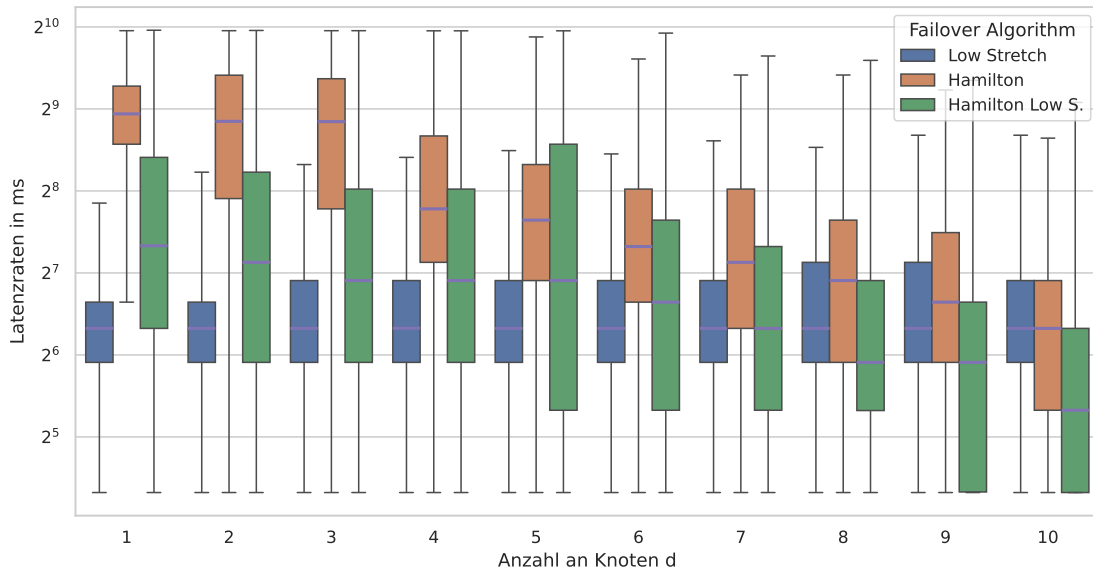


Abbildung 5.8.: Latenzen unter Clusterfailure Node. $p = 0.9 \wedge q = 0.34$. 7×7 Torus.

5. Evaluation

5.2. Durchsatz

Innerhalb dieses Abschnitts folgt die Analyse des Durchsatzes. Dazu sei erwähnt, dass die grafischen Darstellungen der Daten auf die theoretische verfügbare Bandbreite in Summe normiert sind und als Anteile in Hundertstel angegeben werden. Zusätzlich sei angeführt, dass ausschließlich »Towards Destination« im »All to single Destination« Traffic Pattern betrieben worden ist, während die anderen Failure Pattern im »All to All« Traffic Pattern betrieben wurden. Nicht erreichbare Paare werden nicht in der Statistik geführt und somit artbedingt in der Summe als null interpretiert. Im »All to single Destination« Traffic-Pattern wird die Summe der Datenraten der Clients und die des Servers als Messpunkt ausgewertet.

5.2.1. Failure pattern: Edges

Random Edges: Beginnend mit der Studie des Durchsatzes (Abbildung 5.9) ist zuerst wieder bezeichnend, dass die Quoten des Low Stretch Verfahrens gefolgt von denen des Hamilton Low Stretch Verfahrens bedeutend besser sind als die des Hamilton Verfahrens. Insgesamt ist über die Zunahme der Kantenausfälle eine Degression der Bandbreitenausnutzung zu erkennen, mit zusätzlich steigender Standardabweichung, die zum Ende wieder abflacht. Abweichend ist hier allerdings das Hamilton basierte Verfahren, in dem die progressive Standardabweichung erst signifikant ab dem siebten Kantenausfall einsetzt und die tatsächliche Bandbreiten Ausnutzung sogar antizyklisch ansteigt. Dies kann dieses Mal nicht analog zur Latenz begründet werden, da arttypisch die aus Summation nicht erreichbaren Paare mit null geführt werden. Eher ist dies ein Indikator für eine abnehmende Congestion im Netzwerk. Kongruente Phänomene können weniger ausgeprägt auch zwischen der 11 und 13 Iteration bei den anderen beide Verfahren observiert werden. Alleinstehend ebenfalls in der fünften Iteration des Low Stretch Verfahrens. Ansonsten ist bei der achten und neunten Iteration die Spannweite bei den Low Stretch Verfahren besonders signifikant.

5.2.2. Failure pattern: Randome Nodes

Randome Nodes: In der Analyse des Durchsatzes (Abbildung 5.10) sind Parallelen zu den Kantenausfällen zu ziehen. Zuerst fällt aber die insgesamt geringere Ausnutzung der Bandbreite über alle Verfahren auf. Dies lässt sich mit der größeren Topologie und den damit einhergehenden längeren Wegen begründen. Zusätzlich sind mehr Paare in Verwendung. Wieder sind analoge Tendenzen in der Standardabweichung und der gesamten Bandbreitenentwicklung zu observieren. Auffällig ist außerdem der größere Abstand zwischen dem Low Stretch Verfahren und Low Stretch Hamilton Verfahren im Vergleich zu den Kantenausfällen.

5.2.3. Failure pattern: Clustered Failure

Cluster Failure: In Betrachtung des Durchsatzes (Abbildung 5.11 und 5.12) wird wieder ein ähnliches Muster zu anderen Failuren Patterns demonstriert. In der Studie der Step-Variante ist abseits der sechsten Iteration mit erhöhter Standardabweichung bei den Low Stretch Verfahren keine weitere Auffälligkeit zu observieren. Bei dem Node-Derivat ist allein markant, dass die doch relativ große Standardabweichung des Low Stretch Verfahrens im Gegensatz zu den Hamilton Verfahren. Vermutlich ist dies begründet auf der Zufallskomponente der Instanzen der Pattern.

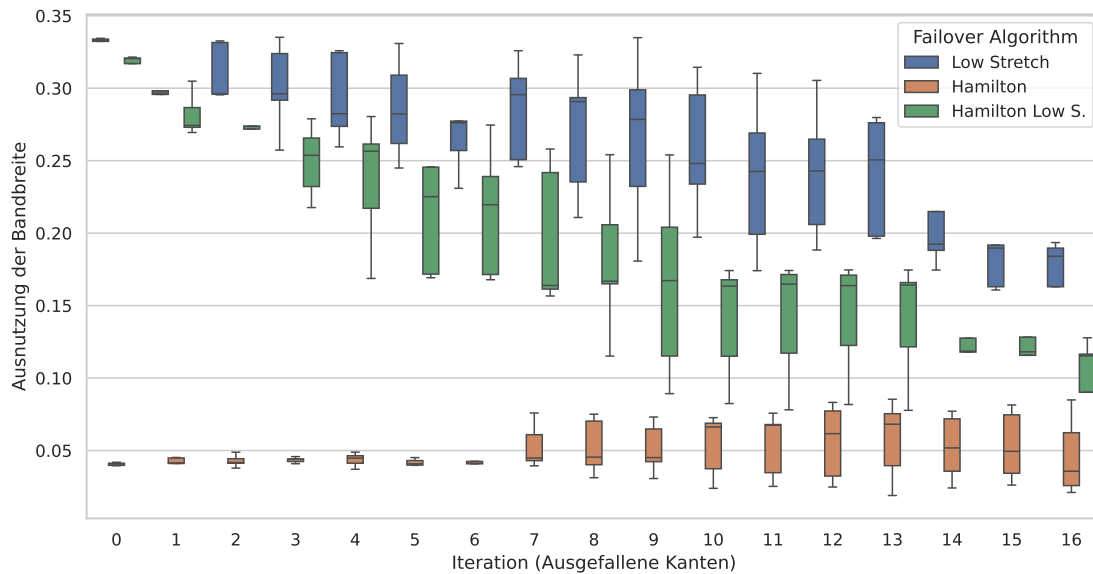


Abbildung 5.9.: Durchsatz unter zufälligen Kanten Ausfällen 5×5 Torus.; All-to-All; pairs= $|9|$

5.2.4. Failure pattern: Towards Destination

Towards Destination: Schließlich lässt sich bei der Auswertung des Durchsatzes (Abbildung 5.13) zuerst feststellen, dass das Pattern seinem Zweck entspricht und die Datenrate mit zunehmenden Kantenausfällen bei allen Verfahren abnimmt. Dies ist wohl durch das Fehlen der direkten Linkkapazitäten aufgrund von Ausfällen zu erklären. Des Weiteren kann beim Low Stretch Verfahren eine geringe Standardabweichung beobachtet werden, sodass davon auszugehen ist, dass die Reihenfolge der Kantenausfälle relativ belanglos ist. Markant ist zudem, dass das Hamilton basierte Verfahren selbst unter nur einem Flow nicht die volle Datenrate ausschöpfen kann, vermutlich aufgrund der Tatsache, dass der Datenverkehr aufgrund der Konstruktion des Verfahrens über einen Knoten fließt (ein Hamilton Zyklus). Zuletzt fällt auf, dass die Datenrate beim Hamilton Verfahren zum Ende des Verlaufs wieder im Median ansteigt. Dies liegt vermutlich daran, dass nur noch der direkte Nachbar eine Verbindung zur Destination hat und so die volle Datenrate des einen Links ausgereizt werden kann. Insgesamt erscheint die Aussagekraft dieses Musters in Bezug auf Failover Verfahren jedoch gering.

5. Evaluation

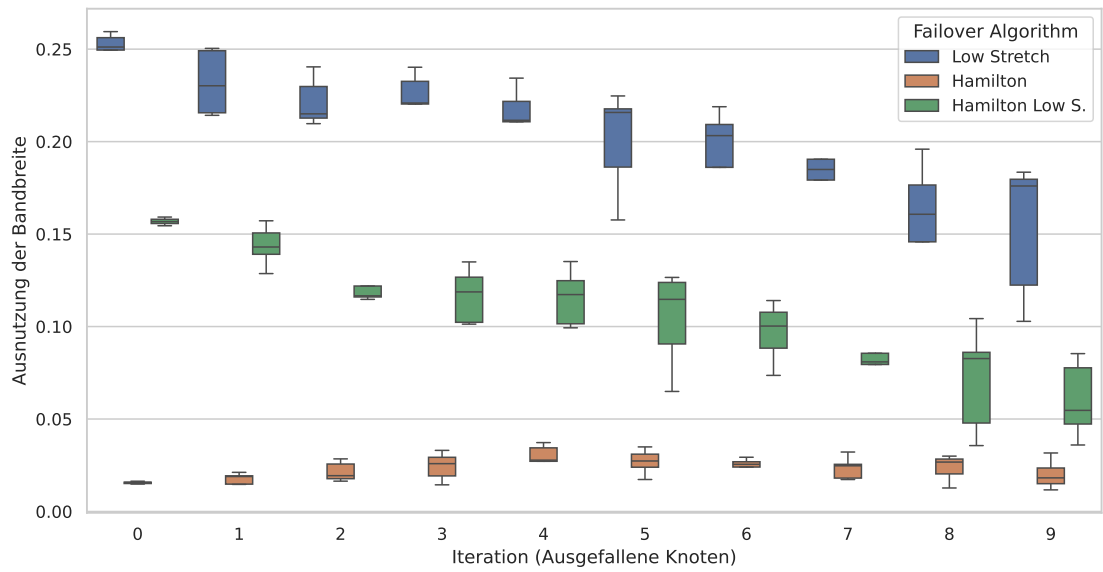


Abbildung 5.10.: Durchsatz unter zufälligen Knoten Ausfällen 7×7 Torus.; All-to-All; pairs= $|20|$

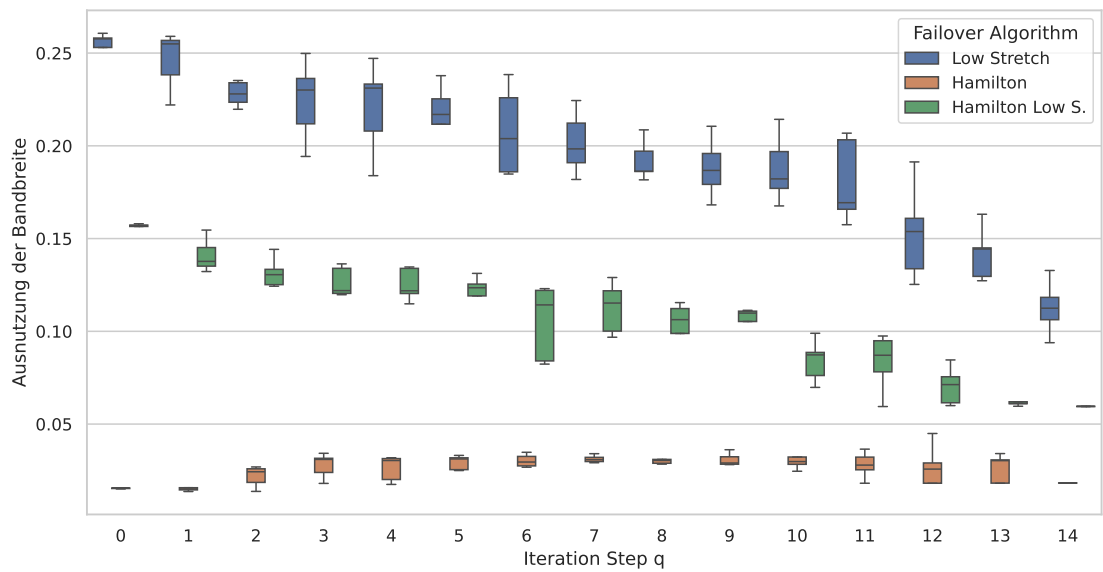


Abbildung 5.11.: Durchsatz unter Clusterfailure mit $p = 0.9$ und $q \in (0.3, 0.9)$. 7×7 Torus.; All-to-All; pairs= $|20|$

5.2. Durchsatz

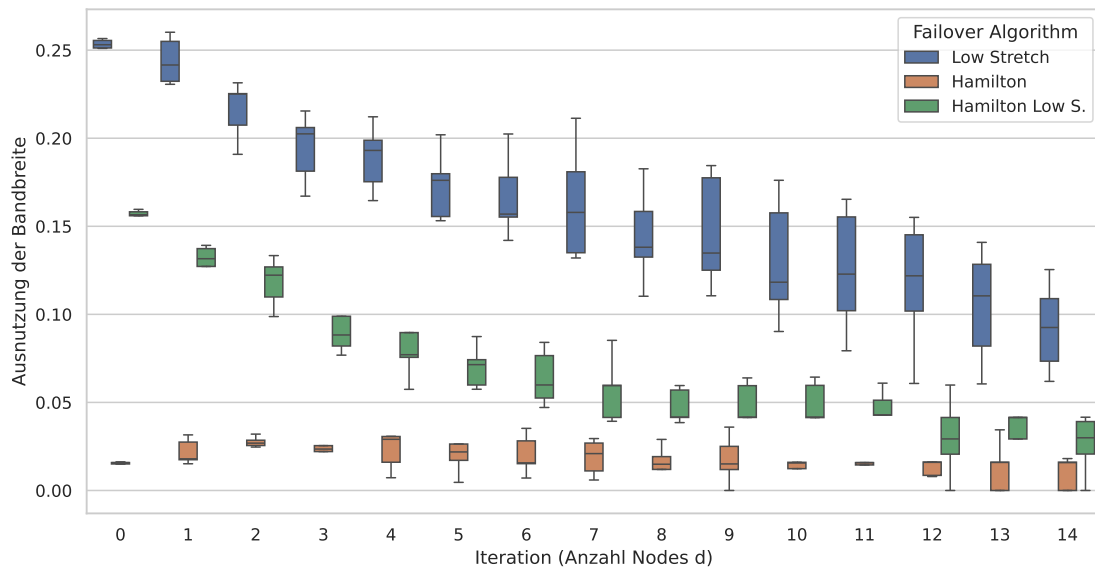


Abbildung 5.12.: Durchsatz unter Clusterfailure mit Node. $p = 0.9 \wedge q = 0.34$. 7×7 Torus.; All-to-All; pairs=|20|

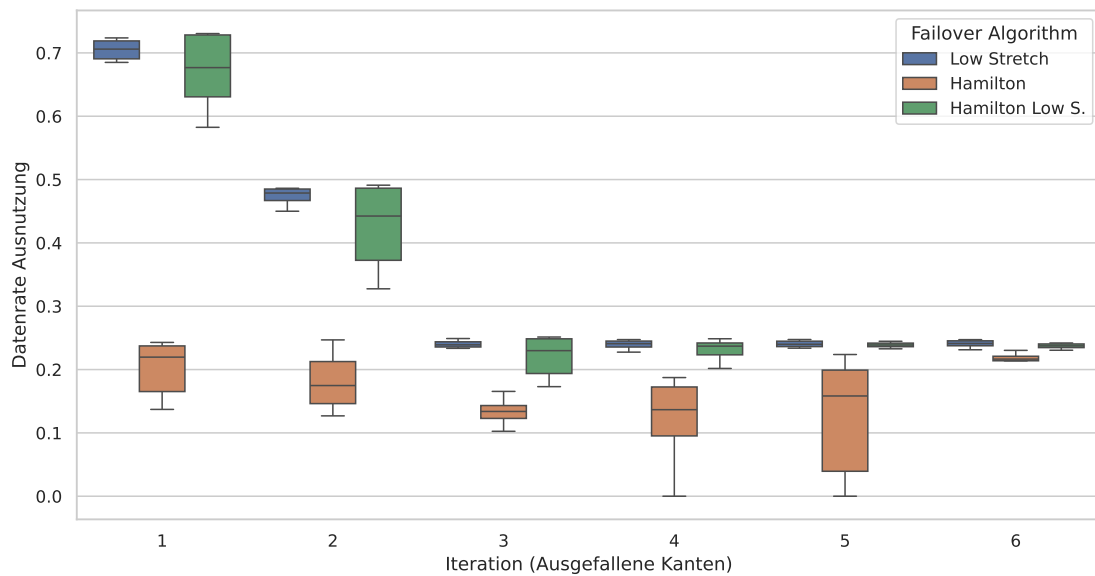


Abbildung 5.13.: Durchsatz unter Towards Destination. 5×5 Torus; All to single Destination

5. Evaluation

5.3. Augmentation: Perfect Resiliency within 2-Hops

Zur Evaluation der Two-Hop Augmentation wird zuerst die einzelne Erreichbarkeit unter dem Ausfallmuster aus Abbildung 5.14 für den Node s zu t getestet. Anschließend erfolgt die Auswertung für die Erreichbarkeit unter zufälligen Kantenausfällen im Vergleich zu den beiden Low Stretch Verfahren.

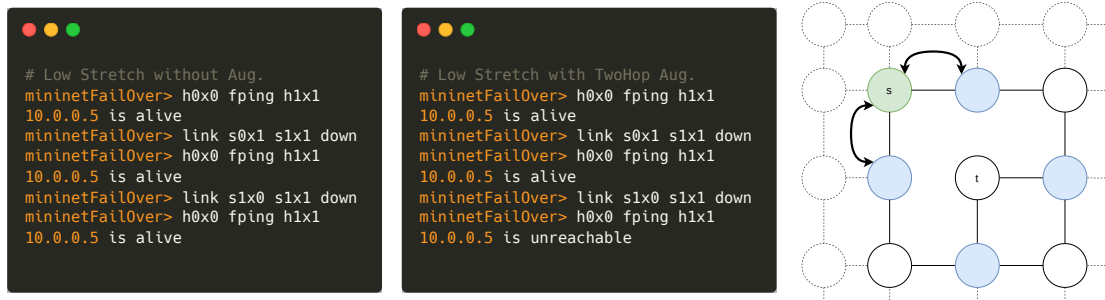


Abbildung 5.14.: Links: Low Stretch; Mitte: Low Stretch mit Aug; Rechts: angewandtes Failure Pattern

Zuerst ist daher das Ausfallmuster zu erläutern. Konkret werden die zwei benachbarten Kanten zum Ziel, die sich auf den kürzesten Wegen von s befinden, deaktiviert. Aus Konstruktion der Augmentation ist die Provokation eines Ausfalles zu erwarten, da die Quelle s über keinen alternativen Übergangsknoten mehr verfügt, der einen Weg zum Ziel innerhalb einer Weglänge von zwei einrichten kann. Aus dem Testlauf (Abbildung 5.14) kann diese Annahme als bestätigt betrachtet werden. Ferner wird demonstriert, dass das gewöhnliche Low Stretch Verfahren sehr wohl in der Lage dazu ist, diese Art der Kantenausfälle zu tolerieren.

Weiter folgt die Betrachtung der Erreichbarkeitsquoten (Abbildung 5.15). Zuerst sei angeführt, dass die Wahl der Intervallgrenzen der Kantenausfälle aus Übersichtsgründen resultiert. Weiterhin sei darauf hingewiesen, dass der Boxplot wieder die nicht zugestellten Pings ausweist (lower is better). Einleitend ist festzustellen, dass in der Gesamtheit das augmentierte Low Stretch Verfahren eine geringfügig schlechtere Quote als das Low Stretch Verfahren produziert, hingegen das augmentierte Hamilton basierte Verfahren eine geringfügig bessere Quote. Eine Anomalie lässt sich nur innerhalb einer Stichprobe zur neunten und zehnten Iteration observieren, da das augmentierte Low Stretch Verfahren etwas besser abschneidet als das gewöhnliche. Ansonsten ist ein analoges Verhalten zu den Kantenausfällen in Abbildung 5.1 zu konstatieren.

5.3. Augmentation: Perfect Resiliency within 2-Hops

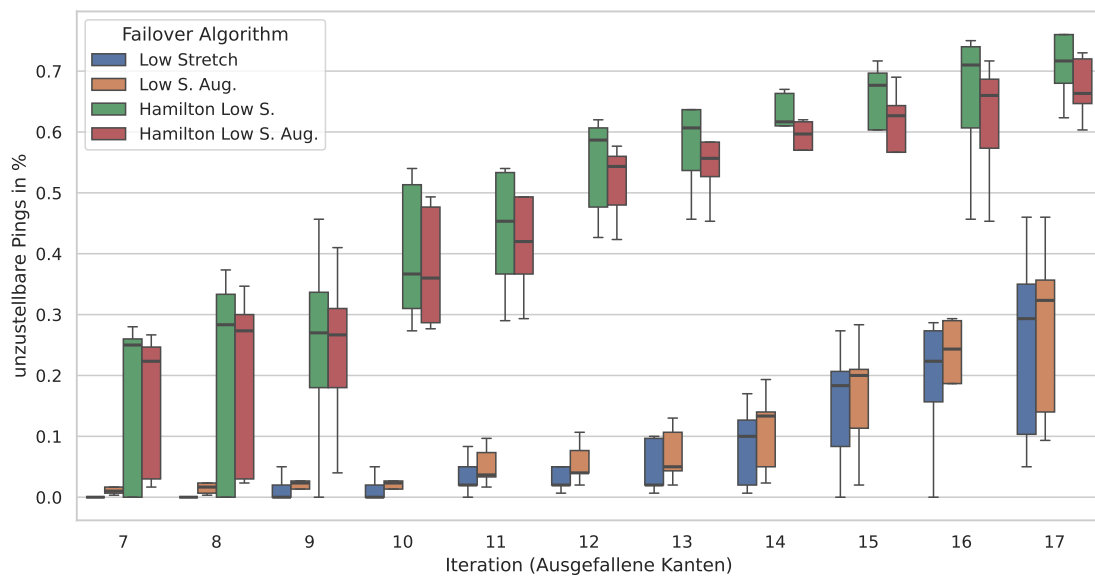


Abbildung 5.15.: Erreichbarkeit und zufälligen Kantenausfällen mit Aug. 5×5 Torus.

6. Diskussion

In diesem Kapitel erfolgt eine Einordnung der in der Evaluation (Kapitel 5) erhobenen Daten.

Zusammenfassend lässt sich zuerst festhalten, dass die Verfahren alle eine Resilienz entsprechend ihrer Forschung[20, 11] aufweisen konnten. Weiterhin demonstriert das Low Stretch Verfahren eine Dominanz in allen Metriken im Vergleich zu den anderen Verfahren, weit abgeschlagen gefolgt von dem Hamilton Low Stretch Verfahren. Bilanziert zeigt sich die Two-Hop Augmentation als nicht erfolgversprechend: Auch wenn das Hamilton Low Stretch Verfahren eine marginale Verbesserung demonstriert, ist konträr die Erreichbarkeitsquote des Low Stretch Verfahrens (leicht) reduziert.

In der Ursachenerforschung bezüglich der Resilienz lässt sich abstrakt eine systemisch überlegene Routenplanung vermuten, die aus der Konstruktion der Algorithmen bedingt ist. Im Detail kann bei hamiltonbasierten Verfahren bereits ein Ausfall von vier Kanten genügen, um die Zyklen hinreichend zu unterbrechen, sodass erhebliche Verwerfungen zu beobachten sind, speziell, wenn das Paket nach Erreichen der Rückrichtung des zweiten Zyklus verworfen wird (Referenz Konstruktion). In der konkreten Implementierung wird dann aber der erste Zyklus als Hail Mary (abermals) probiert, mit der Folge, dass sich ein etwas differentes Verhalten zeigt. Unter tieferer Analyse der zufälligen Kantenausfälle der fünften und sechsten Iteration zeigt sich innerhalb der zweiten Stichprobe eine Ausfallquote von über 40%. Aufschlussreich ist hierbei die Position der ausgefallenen Kanten im Verhältnis zu den beiden Hamilton Zyklen, denn es offenbart sich, dass die Kantenausfälle gleichverteilt auf beiden Zyklen liegen und aber vor allem auch, dass die Entfernung innerhalb der Zyklen relativ gesehen kurz ist. Wesentlich ist aber eher, dass ein ganzes Cluster von Knoten innerhalb der Failover Entscheidung bedingt durch die Positionierung der Zyklen eine Schleife induzieren und so die Pakete nicht zugestellt werden können. Für einen Gegner interessante Positionen scheinen daher wohl die »Kurven« der Zyklen zu sein bei gleichverteilter Verteilung auf beiden Zyklen. Letzteres scheint unter Betrachtung der Steigerung der Ausfallraten der anderen Stichproben im Verlauf besonders ausschlaggebend zu sein. Spannend ist daher auch der Vergleich mit den Knotenausfällen, wo die Positionierung der Knoten einen Unterschied macht, wenngleich längst nicht so »erfolgreich« wie die Kantenausfälle. Insgesamt scheint aber auch eine gewisse räumliche Nähe von Vorteil zu sein sowie eine Platzierung nahe von Kurven. Somit könnte vielleicht die »linke« Seite (des Schaubildes; Abbildung 3.1) der Topologie als verwundbarer vermutet werden, aber eine derartige Aussage würde zur Bestätigung bedeutend mehr Nachforschungen erfordern.

In Betrachtung des Hamilton Low Stretch Verfahren im Vergleich scheint es im Grundsatz von Vorteil zu sein kurze Routen zu wählen, weil die Wahrscheinlichkeit auf eine ausgefallene Kante zu treffen mit der Weglänge steigt. Zusätzlich wird der gleiche (schon funktionierende) Weg zurück gewählt, wenn auf dem Hinweg keine ausfallende Kante angetroffen wurde (wieder der kürzeste Hamilton Zyklen in gegensätzlicher Richtung). Verbleibend lässt sich die Analyse des Hamilton Verfahren übertragen.

In Deliberation des Low Stretch Verfahren kann zuerst der tatsächlich fast immer optimale Weg diesen Vorteil voll auskosten, insbesondere da das Umschiffen von ausfallenden Kanten keine vollständige Wanderung durch die Topologie auslöst. Des Weiteren ergeben sich mittelbar aus der Konstruktion keine Kanten, die durch einen Ausfall global Reroutingaktion provozieren, da jedes Ziel über einen eigenen

6. Diskussion

Shortest Path Tree verfügt. Ebenso ist es daher nicht gelungen ein (rudimentäres) gezieltes Ausfallmuster abseits bekannter Methodik zu isolieren. In Analyse der Ausreißer, der hinteren Iteration der Kantenausfälle ließe sich nur identifizieren, dass ganze Knotencluster der Topologie isoliert oder nur noch spärlich angebunden waren. Dies scheint auch eine Kausalität für die unterschiedlichen Stichproben zu sein.

Zur Studie der Latenz ist die Diagnose eindeutig. Die Latenz kann gleich gesetzt werden mit der Weglänge und ist hinreichend bekannt. Bei dem normalen Hamilton Verfahren bewandert das Paket einmal alle Knoten und im Falle eines Ausfalls ggf. weniger, oder aber mehr je nach Position, analog, dass low Stretch Hamilton Verfahren (mit Divergenz des Verhaltens ohne Ausfälle). Im Kontrast dazu wählt das Low Stretch Verfahren nahezu optimale Routen auch im Falle eines Ausfalls. Trotzdem kann mit zunehmenden Ausfällen aufgrund steigender Weglänge auch ein Anstieg der Latenz beobachtet werden. Zu erwähnen ist hier wohl noch, dass die Abnahme der Latenzen zum Ende innerhalb der gesamten Population auf nicht erreichbare Paare zurückzuführen ist. Der Durchsatz ist da schon etwas aufschlussreicher. Bei dem Hamilton Verfahren zeigt sich initial eine desaströse Ausnutzung der Bandbreite, die mit Zunahme der Ausfälle tatsächlich wieder zunimmt. Vermutlich, weil die gesamte Kongestion im Netzwerk aufgrund nicht erreichbarer Paare sinkt und ggf. beide Hamilton Zyklen in beide Richtung ausgenutzt werden. Letzteres ist wohl auch der direkte Vorteil des Hamilton Low Stretch Verfahren mit zugleich ebenfalls kürzeren Routen.

Zusätzlich ist zu erwähnen, dass die Ressourcen eines Netzwerks endlich sind und mit zunehmender Weglänge natürlich die Ausnutzung steigt, somit ist das Low Stretch Verfahren wie im Vorteil. Dies findet sich auch in der Durchsatzmessung wieder. Auch ist architekturbedingt die Kanten Ausnutzung eher opportun.

Schließlich zur Augmentation. Wie in der Demonstration ist die Mechanik alsbald überfordert, sofern alle alternativen Wege der Länge zwei fehlgeschlagen sind. In den Grenzen der Topologie sind das auch nur zwei Wege. Letztlich ist die Auswirkung jedoch gering, da die Anzahl der Knotenpaare, auf die überhaupt Einfluss genommen wird, durch die Distanz begrenzt ist.

7. Abschluss

In diesem Kapitel erfolgt erst innerhalb der Rekapitulation (Sektion 7.1) eine Reflexion im Rahmen des Rückblicks auf die Arbeit. Anschließend werden im Ausblick (Sektion 7.2) Impulse für die weitere Forschung präsentiert. Das Fazit (Sektion 7.3) bildet den Ausklang.

7.1. Rekapitulation

Rekapitulierend wurde im Verlauf dieser Arbeit das Thema des Fast Failover Routings innerhalb der Topologie eines Torus in Mininet in Komposition mit OpenVSwitch untersucht. Im besonderen Fokus stand dabei die Realisierung mehrerer Fast Fast Failover Verfahren mittels OpenFlow innerhalb der Fast Failover Gruppe und die anschließende Evaluation gegenüber verschiedener Metriken. Innerhalb der Einleitung und Grundlagen (Kapitel 1 und 2) wurde das theoretische Fundament gegossen und der Stand der Forschung präsentiert. Anschließend liefern die verwandten Arbeiten (Kapitel 3) einen vertiefenden Einblick in relevante Publikation und Literatur. Zusätzlich wurde schwerpunktmäßig auch die Forschung der betrachteten Fast Failover Verfahren präsentiert, darunter speziell auch die Funktionsweise. Hierauf wurde im Kapitel der Methodik und Implementierung (Kapitel 4) die konkrete Umsetzung des Test-Frameworks dargestellt und die Herangehensweise der Evaluation ausführlich beleuchtet. Zu akzentuieren ist vordergründig die Realisierung der Fast Failover Verfahren mittels OpenFlow und der Fast Failover Gruppen in Mininet innerhalb des Test-Frameworks. Nachfolgenden wurden im Kapitel der Evaluation und Diskussion (Kapitel 5 und 6) die Resultate ausführlich dargestellt. Resümierend kann als Ergebnis zusammengefasst werden, dass das Low Stretch Verfahren insbesondere aufgrund der Routenqualität, aber auch architektonisch überlegen ist. Weiterhin ist eine Realisierung in OpenFlow praktikabel, sofern die Einschränkung kein unüberwindliches Hindernis hinsichtlich der Kapazitäten eines Switches darstellen. In der eigentlichen Evaluation ist dennoch der unmittelbare Nutzen aus den Fast Failover Gruppe begrenzt, denn denkbar wären auch andere, möglicherweise einfachere Flow Steuerungen, die ähnliche Ergebnisse produzieren könnten. Jedoch könnte dies die praktische Umsetzbarkeit möglicherweise nicht ausreichend demonstrieren. Zuletzt hat sich die Two-Hop-Augmentation als nicht zweckdienlich herausgestellt. Demgegenüber hat der Low Stretch Hamilton Algorithmus opportun im Verhältnis zu Hamilton Verfahren abgeschlossen.

7.2. Ausblick

Auf Basis der gewonnenen Erkenntnisse ergeben sich mehrere Ansatzpunkte für die zukünftige Forschung. Eine zentrale Fragestellung ergibt sich aus der Generalisierung der Ergebnisse auf andere artverwandte Topologien wie Hypercubes, Clos Netzwerke oder Multi dimensionale Grids insbesondere, da Foerster et al.[20] auch Stretch optimale Verfahren beschreibt. Weiterhin ließe ggf. das bekannte Verfahren auf einen diagonalen Torus übertragen[63]. Des Weiteren könnte auch eine Studie unter Einsatz größere

7. Abschluss

Stichproben und erweiterter Ausfallmodelle die Generalisierbarkeit der Ergebnisse verbessern. Zu diesem Zweck könnte realer Traffic außerhalb synthetischer Traffic Generatoren eingesetzt werden in Kombination mit komplexeren Ausfallmustern. Ferner wäre auch ein Vergleich zu Failover Algorithmen interessant, die die Fähigkeit von OpenFlow oder P4 voll ausnutzen und zusätzlich beispielgebend Paket Modifikation einsetzen.

Schließlich könnte ein weiterer Aspekt der Forschung innerhalb einer vertiefenden Analyse von gezielten Ausfallmustern für die eingesetzten Fast Failover Verfahren liegen.

7.3. Fazit

Abschließend bleibt als Fazit festzuhalten, dass die Arbeit einen Beitrag zur Erforschung des Einsatzes von beschriebenen Fast Failover Verfahren innerhalb realer Netzwerke leistet und eine Demonstration des Potenzials von OpenFlow und Fast Failovergruppen in OpenVSwitch darstellt. Weiterhin zeigt sich aufgrund der erhobenen Daten die Signifikanz der Qualität der Fast Failover Routen und deshalb auch die praktische Überlegenheit des Low Stretch Verfahrens. Ferner wurde aufgezeigt, dass eine Evaluation der Performance von (Fast Failover) Routing Verfahren in Ausfallszenarien durchaus robust innerhalb virtueller Netzwerke erfolgen kann, ohne auf abstrakte Softwarekonstruktion zurückzugreifen. Dafür wurde ein entsprechendes Framework entwickelt. Überdies kann eine (experimentelle) Auswertung abseits theoretischer Möglichkeiten die erweiterten Leistungscharakteristiken eines Verfahrens aufdecken. Es obliegt nun, diese Impulse aufzugreifen.

Literaturverzeichnis

- [1] 1981. *Internet Control Message Protocol*. Technical Report 792. <https://doi.org/10.17487/RFC0792>
- [2] 2024. Torusbasiertes Fast-Failover Routing. <https://github.com/Xceldur/Torusbasiertes-Fast-Failover-Routing> Accessed: 2024-06-18.
- [3] Mohammad Alizadeh, Albert G. Greenberg, David A. Maltz, Jitendra Padhye, Parveen Patel, Balaji Prabhakar, Sudipta Sengupta, and Murari Sridharan. 2010. Data center TCP (DCTCP). In *Proceedings of the ACM SIGCOMM 2010 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications, New Delhi, India, August 30 -September 3, 2010*, Shivkumar Kalyanaraman, Venkata N. Padmanabhan, K. K. Ramakrishnan, Rajeev Shorey, and Geoffrey M. Voelker (Eds.). ACM, 63–74. <https://doi.org/10.1145/1851182.1851192>
- [4] Alia Atlas, George Swallow, and Ping Pan. 2005. *Fast Reroute Extensions to RSVP-TE for LSP Tunnels*. Technical Report 4090. RFC Editor. <https://doi.org/10.17487/RFC4090>
- [5] Vaibhav Bajpai, Anna Brunström, Anja Feldmann, Wolfgang Kellerer, Aiko Pras, Henning Schulzrinne, Georgios Smaragdakis, Matthias Wählisch, and Klaus Wehrle. 2019. The Dagstuhl Beginners Guide to Reproducibility for Experimental Networking Research. *CoRR* abs/1902.02165 (2019). arXiv:1902.02165 <http://arxiv.org/abs/1902.02165>
- [6] Cüneyt Bayilmis, M. Ali Ebleme, Ünal Çavusoglu, Kerem Küçük, and Abdullah Sevin. 2022. A survey on communication protocols and performance evaluations for Internet of Things. *Digit. Commun. Networks* 8, 6 (2022), 1094–1104. <https://doi.org/10.1016/J.DCAN.2022.03.013>
- [7] Soeren Becker, Tobias Pfandzelter, Nils Japke, David Bermbach, and Odej Kao. 2022. Network Emulation in Large-Scale Virtual Edge Testbeds: A Note of Caution and the Way Forward. In *IEEE International Conference on Cloud Engineering, IC2E 2022, Pacific Grove, CA, USA, September 26-30, 2022*. IEEE, 1–7. <https://doi.org/10.1109/IC2E55432.2022.00007>
- [8] Michael Borokhovich and Stefan Schmid. 2013. How (Not) to Shoot in Your Foot with SDN Local Fast Failover: A Load-Connectivity Tradeoff. *CoRR* abs/1309.3150 (2013). arXiv:1309.3150 <http://arxiv.org/abs/1309.3150>
- [9] Giorgio Calarco and Maurizio Casoni. 2013. On the effectiveness of Linux containers for network virtualization. *Simul. Model. Pract. Theory* 31 (2013), 169–185. <https://doi.org/10.1016/J.SIMPAT.2012.11.007>
- [10] Tao Chen, Xiaofeng Gao, and Guihai Chen. 2016. The features, hardware, and architectures of data center networks: A survey. *J. Parallel Distributed Comput.* 96 (2016), 45–74. <https://doi.org/10.1016/J.JPDC.2016.05.009>

Literaturverzeichnis

- [11] Marco Chiesa, Andrei Gurtov, Aleksander Mądry, Slobodan Mitrović, Ilya Nikolaevskiy, Aurojit Panda, Michael Schapira, and Scott Shenker. 2016. Exploring the Limits of Static Failover Routing. arXiv:1409.0034 [cs.NI]
- [12] Marco Chiesa, Andrzej Kamisinski, Jacek Rak, Gábor Rétvári, and Stefan Schmid. 2021. A Survey of Fast-Recovery Mechanisms in Packet-Switched Networks. *IEEE Commun. Surv. Tutorials* 23, 2 (2021), 1253–1301. <https://doi.org/10.1109/COMST.2021.3063980>
- [13] Marco Chiesa, Ilya Nikolaevskiy, Slobodan Mitrovic, Andrei V. Gurtov, Aleksander Madry, Michael Schapira, and Scott Shenker. 2017. On the Resiliency of Static Forwarding Tables. *IEEE/ACM Trans. Netw.* 25, 2 (2017), 1133–1146. <https://doi.org/10.1109/TNET.2016.2619398>
- [14] Marco Chiesa, Ilya Nikolaevskiy, Slobodan Mitrovic, Aurojit Panda, Andrei V. Gurtov, Aleksander Madry, Michael Schapira, and Scott Shenker. 2016. The quest for resilient (static) forwarding tables. In *35th Annual IEEE International Conference on Computer Communications, INFOCOM 2016, San Francisco, CA, USA, April 10-14, 2016*. IEEE, 1–9. <https://doi.org/10.1109/INFOCOM.2016.7524552>
- [15] Renzo Davoli. 2017. VXVDEX: Internet of threads and networks of namespaces. In *IEEE International Conference on Communications, ICC 2017, Paris, France, May 21-25, 2017*. IEEE, 1–6. <https://doi.org/10.1109/ICC.2017.7996595>
- [16] Paul Emmerich, Daniel Raumer, Sebastian Gallenmüller, Florian Wohlfart, and Georg Carle. 2018. Throughput and Latency of Virtual Switching with Open vSwitch: A Quantitative Analysis. *J. Netw. Syst. Manag.* 26, 2 (2018), 314–338. <https://doi.org/10.1007/S10922-017-9417-0>
- [17] Joan Feigenbaum, Brighten Godfrey, Aurojit Panda, Michael Schapira, Scott Shenker, and Ankit Singla. 2012. On the Resilience of Routing Tables. *CoRR* abs/1207.3732 (2012). arXiv:1207.3732 <http://arxiv.org/abs/1207.3732>
- [18] Klaus-Tycho Foerster, Juho Hirvonen, Yvonne-Anne Pignolet, Stefan Schmid, and Gilles Trédan. 2021. On the Feasibility of Perfect Resilience with Local Fast Failover. In *2nd Symposium on Algorithmic Principles of Computer Systems, APOCS 2020, Virtual Conference, January 13, 2021*, Michael Schapira (Ed.). SIAM, 55–69. <https://doi.org/10.1137/1.9781611976489.5>
- [19] Klaus-Tycho Foerster, Juho Hirvonen, Yvonne-Anne Pignolet, Stefan Schmid, and Gilles Trédan. 2022. On the Price of Locality in Static Fast Rerouting. In *52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN 2022, Baltimore, MD, USA, June 27-30, 2022*. IEEE, 215–226. <https://doi.org/10.1109/DSN53405.2022.00032>
- [20] Klaus-Tycho Foerster, Yvonne-Anne Pignolet, Stefan Schmid, and Gilles Trédan. 2018. Local Fast Failover Routing With Low Stretch. *Comput. Commun. Rev.* 48, 1 (2018), 35–41. <https://doi.org/10.1145/3211852.3211858>
- [21] Klaus-Tycho Foerster, Yvonne-Anne Pignolet, Stefan Schmid, and Gilles Trédan. 2019. CASA: Congestion and Stretch Aware Static Fast Rerouting. In *2019 IEEE Conference on Computer Communications, INFOCOM 2019, Paris, France, April 29 - May 2, 2019*. IEEE, 469–477. <https://doi.org/10.1109/INFOCOM.2019.8737438>

- [22] Haryadi S. Gunawi, Mingzhe Hao, Riza O. Suminto, Agung Laksono, Anang D. Satria, Jeffry Adityatama, and Kurnia J. Eliazar. 2016. Why Does the Cloud Stop Computing? Lessons from Hundreds of Service Outages. In *Proceedings of the Seventh ACM Symposium on Cloud Computing, Santa Clara, CA, USA, October 5-7, 2016*, Marcos K. Aguilera, Brian Cooper, and Yanlei Diao (Eds.). ACM, 1–16. <https://doi.org/10.1145/2987550.2987583>
- [23] Nikhil Handigol, Brandon Heller, Vimalkumar Jeyakumar, Bob Lantz, and Nick McKeown. 2012. Reproducible network experiments using container-based emulation. In *Conference on emerging Networking Experiments and Technologies, CoNEXT '12, Nice, France - December 10 - 13, 2012*, Chadi Barakat, Renata Teixeira, K. K. Ramakrishnan, and Patrick Thiran (Eds.). ACM, 253–264. <https://doi.org/10.1145/2413176.2413206>
- [24] Benjamin Hardin, Douglas Comer, and Adib Rastegarnia. 2023. On the Unreliability of Network Simulation Results FROM Mininet and iPerf. *International Journal of Future Computer and Communication* 12, 1 (March 2023), 5–13. <https://doi.org/10.18178/ijfcc.2023.12.1.596>
- [25] David Harrington and David Levi. 2005. *Definitions of Managed Objects for Bridges with Rapid Spanning Tree Protocol*. Technical Report 4318. <https://doi.org/10.17487/RFC4318>
- [26] Brandon Heller. 2013. *Reproducible network research with high-fidelity emulation*. Ph.D. Dissertation. Stanford University, USA. <https://searchworks.stanford.edu/view/10164606>
- [27] Stephen Hemminger. 2005. Network emulation with NetEm. *Linux Conf Au* (05 2005).
- [28] Paco Hope. 2002. Using Jails in FreeBSD for Fun and Profit. *login Usenix Mag.* 27, 3 (2002). <https://www.usenix.org/publications/login/june-2002-volume-27-number-3/using-jails-freebsd-fun-and-profit>
- [29] Christian Hopps. 2000. *Analysis of an Equal-Cost Multi-Path Algorithm*. Technical Report 2992. RFC Editor. <https://doi.org/10.17487/RFC2992>
- [30] Roland J. Schemers III, RL "BobMorgan, and David Papp. 1992. *fping*. <http://www.fping.com> Konzept und Versionen 1.x von Roland J. Schemers III, Versionen 2.x von RL "BobMorgan, Versionen 2.3x und höher von David Papp. Netzwerkdiagnosetool zum schnellen Senden von ICMP-Echo-Anfragen an mehrere Hosts..
- [31] IP utilities project. 2024. *ping man page*. The Open Group. <https://man7.org/linux/man-pages/man8/ping.8.html>.
- [32] The iperf2 Project. 2024. *iperf(1) — iperf - The TCP/UDP Bandwidth Measurement Tool*. The iperf2 Project. <https://manpages.debian.org/testing/iperf/iperf.1.en.html> Accessed: 2024-06-01.
- [33] Audrius Jurgelionis, Jukka-Pekka Laulajainen, Matti Hirvonen, and Alf Inge Wang. 2011. An Empirical Study of NetEm Network Emulation Functionalities. In *Proceedings of 20th International Conference on Computer Communications and Networks, ICCCN 2011, Maui, Hawaii, USA, July 31 - August 4, 2011*, Haohong Wang, Jin Li, George N. Rouskas, and Xiaobo Zhou (Eds.). IEEE, 1–6. <https://doi.org/10.1109/ICCCN.2011.6005933>

Literaturverzeichnis

- [34] A. B. Katok and Vaughn Climenhaga. 2008. *Lectures on surfaces : (almost) everything you wanted to know about them*. American Mathematical Society ; Mathematics Advanced Study Semesters, Providence, R.I.; [Philadelphia].
- [35] Nauman Khan, Rosli Bin Salleh, Anis Koubaa, Zahid Khan, Muhammad Khurram Khan, and Ihsan Ali. 2023. Data plane failure and its recovery techniques in SDN: A systematic literature review. *J. King Saud Univ. Comput. Inf. Sci.* 35, 3 (2023), 176–201. <https://doi.org/10.1016/J.JKSUCI.2023.02.001>
- [36] Kiran Koushik, Riza Cetin, and Thomas Nadeau. 2011. *Multiprotocol Label Switching (MPLS) Traffic Engineering Management Information Base for Fast Reroute*. Technical Report 6445. RFC Editor. <https://doi.org/10.17487/RFC6445>
- [37] Andy Lawrence and Lenny Simon. 2023. *Annual outage analysis 2023*. Annual Report. Uptime Institute.
- [38] Daniel Lezcano, Christian Brauner, Serge Halryn, and Stéphane Graber. n.d.. *Linux Containers (LXC) man page*. Retrieved 2024-04-17 from <https://linuxcontainers.org/lxc/manpages/man7/lxc.7.html>
- [39] Dawn Li, Bruce A. Cole, Phil Morton, and Tony Li. 1998. *Cisco Hot Standby Router Protocol (HSRP)*. Technical Report 2281. <https://doi.org/10.17487/RFC2281>
- [40] Wenwei Li, Dafang Zhang, Jinmin Yang, and Gaogang Xie. 2007. On evaluating the differences of TCP and ICMP in network measurement. *Comput. Commun.* 30, 2 (2007), 428–439. <https://doi.org/10.1016/J.COMCOM.2006.09.015>
- [41] Roderick J. A. Little and Donald B. Rubin. 2002. *Statistical Analysis with Missing Data*. John Wiley & Sons, Hoboken, NJ. <https://doi.org/10.1002/9781119013563>
- [42] Marc Manzano, Eusebi Calle, Victor Torres-Padrosa, Juan Segovia, and David Harle. 2013. Endurance: A new robustness measure for complex networks under multiple failure scenarios. *Comput. Networks* 57, 17 (2013), 3641–3653. <https://doi.org/10.1016/J.COMNET.2013.08.011>
- [43] Nick McKeown, Thomas E. Anderson, Hari Balakrishnan, Guru M. Parulkar, Larry L. Peterson, Jennifer Rexford, Scott Shenker, and Jonathan S. Turner. 2008. OpenFlow: enabling innovation in campus networks. *Comput. Commun. Rev.* 38, 2 (2008), 69–74. <https://doi.org/10.1145/1355734.1355746>
- [44] Ahlem Menaceur, Hamza Drid, and Mohamed Rahouti. 2023. Fault Tolerance and Failure Recovery Techniques in Software-Defined Networking: A Comprehensive Approach. *J. Netw. Syst. Manag.* 31, 4 (2023), 83. <https://doi.org/10.1007/S10922-023-09772-X>
- [45] Justin Meza, Tianyin Xu, Kaushik Veeraraghavan, and Onur Mutlu. 2018. A Large Scale Study of Data Center Network Reliability. In *Proceedings of the Internet Measurement Conference 2018, IMC 2018, Boston, MA, USA, October 31 - November 02, 2018*. ACM, 393–407. <https://dl.acm.org/citation.cfm?id=3278566>

- [46] John Moy. 1998. *OSPF Version 2*. Technical Report 2328. <https://doi.org/10.17487/RFC2328>
- [47] David Muelas, Javier Ramos, and Jorge E. López de Vergara. 2018. Assessing the Limits of Mininet-Based Environments for Network Experimentation. *IEEE Netw.* 32, 6 (2018), 168–176. <https://doi.org/10.1109/MNET.2018.1700277>
- [48] Stephen Nadas. 2010. *Virtual Router Redundancy Protocol (VRRP) Version 3 for IPv4 and IPv6*. Technical Report 5798. <https://doi.org/10.17487/RFC5798>
- [49] Sebastian Neumayer, Gil Zussman, Reuven Cohen, and Eytan H. Modiano. 2011. Assessing the Vulnerability of the Fiber Infrastructure to Disasters. *IEEE/ACM Trans. Netw.* 19, 6 (2011), 1610–1623. <https://doi.org/10.1109/TNET.2011.2128879>
- [50] Open Networking Foundation. 2015. *OpenFlow Switch Specification Version 1.5.1*. Technical Report. Open Networking Foundation. Retrieved 2024-02-31 from <https://opennetworking.org/wp-content/uploads/2014/10/openflow-switch-v1.5.1.pdf>
- [51] Open vSwitch Project. 2024. *Open vSwitch: An Open Source Production Quality, Multilayer Virtual Switch*. <https://www.openvswitch.org>
- [52] Jelena Pesic, Julien Meuric, Esther Le Rouzic, Laurent Dupont, and Michel Morvan. 2012. Proactive failure detection for WDM carrying IP. In *Proceedings of the IEEE INFOCOM 2012, Orlando, FL, USA, March 25-30, 2012*, Albert G. Greenberg and Kazem Sohraby (Eds.). IEEE, 2971–2975. <https://doi.org/10.1109/INFCOM.2012.6195740>
- [53] Ben Pfaff, Justin Pettit, Teemu Koponen, Ethan J. Jackson, Andy Zhou, Jarno Rajahalme, Jesse Gross, Alex Wang, Joe Stringer, Pravin Shelar, Keith Amidon, and Martín Casado. 2015. The Design and Implementation of Open vSwitch. *login Usenix Mag.* 40, 2 (2015). <https://www.usenix.org/publications/login/apr15/pfaff>
- [54] Diana Andreea Popescu. 2019. *Latency-driven performance in data centres*. Ph.D. Dissertation. University of Cambridge, UK. <https://doi.org/10.17863/CAM.38843>
- [55] FreeBSD Documentation Project. n.d.. *The FreeBSD Handbook: jails*. Retrieved 2024-05-01 from <https://docs.freebsd.org/de/books/handbook/jails/>
- [56] Arjun Roy, Hongyi Zeng, Jasmeet Bagga, and Alex C. Snoeren. 2017. Passive Realtime Datacenter Fault Detection and Localization. *login Usenix Mag.* 42, 3 (2017). <https://www.usenix.org/publications/login/fall2017/roy>
- [57] Khaled Salah, A. Manea, Sherali Zeadally, and José M. Alcaraz Calero. 2011. On Linux starvation of CPU-bound processes in the presence of network I/O. *Comput. Electr. Eng.* 37, 6 (2011), 1090–1105. <https://doi.org/10.1016/J.COMPELECENG.2011.07.001>
- [58] Salvatore Sanfilippo. 2024. *hping3 — send (almost) arbitrary TCP/IP packets to network hosts*. Retrieved 2024-05-31 from <https://manpages.debian.org/unstable/hping3/hping3.8.en.html>

Literaturverzeichnis

- [59] Oliver Schweiger, Klaus-Tycho Foerster, and Stefan Schmid. 2021. Improving the Resilience of Fast Failover Routing: TREE (Tree Routing to Extend Edge disjoint paths). In *ANCS '21: Symposium on Architectures for Networking and Communications Systems, Lafayette, IN, USA, December 13 - 16, 2021*. ACM, 1–7. <https://doi.org/10.1145/3493425.3502747>
- [60] Henk Smit and Naiming Shen. 2004. *Calculating Interior Gateway Protocol (IGP) Routes Over Traffic Engineering Tunnels*. Technical Report 3906. RFC Editor. <https://doi.org/10.17487/RFC3906>
- [61] William Stallings. 2015. *Foundations of Modern Networking: SDN, NFV, QoE, IoT, and Cloud*. Addison-Wesley Professional.
- [62] Andrew Tanenbaum, David Wetherall, and Nick Feamster. 2021. *Computer Networks Global Edition*. Pearson Deutschland. 944 pages. <https://elibrary.pearson.de/book/99.150005/9781292374017>
- [63] K. Wendy Tang and Sanjay A. Padubidri. 1994. Diagonal and Toroidal Mesh Networks. *IEEE Trans. Computers* 43, 7 (1994), 815–826. <https://doi.org/10.1109/12.293260>
- [64] Mininet Team. 2024. Mininet: An Instant Virtual Network on your Laptop (or other PC). <https://mininet.org>.
- [65] The iperf2 Project. 2024. *iperf2*. <https://sourceforge.net/projects/iperf2/>
- [66] The Linux man-pages project. n.d.. *Network Namespaces*. The Linux man-pages project. Retrieved 2024-05-13 from https://man7.org/linux/man-pages/man7/network_namespaces.7.html
- [67] Pang-Wei Tsai, Francesco Piccialli, Chun-Wei Tsai, Mon-Yen Luo, and Chu-Sing Yang. 2017. Control frameworks in network emulation testbeds: A survey. *J. Comput. Sci.* 22 (2017), 148–161. <https://doi.org/10.1016/J.JOCS.2017.03.003>
- [68] Daniel Turner, Kirill Levchenko, Alex C. Snoeren, and Stefan Savage. 2010. California fault lines: understanding the causes and impact of network failures. In *Proceedings of the ACM SIGCOMM 2010 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications, New Delhi, India, August 30–September 3, 2010*, Shivkumar Kalyanaraman, Venkata N. Padmanabhan, K. K. Ramakrishnan, Rajeev Shorey, and Geoffrey M. Voelker (Eds.). ACM, 315–326. <https://doi.org/10.1145/1851182.1851220>
- [69] Lutz Volkmann. 1996. *Hamiltonsche Graphen*. Springer Vienna, Vienna, 76–94. https://doi.org/10.1007/978-3-7091-9449-2_4
- [70] Open vSwitch Project. [n.d.]. *ovs-ofctl(8): Open vSwitch switch interface (ofproto) management tool*. Debian. <https://manpages.debian.org/unstable/openvswitch-common/ovs-ofctl.8.en.html>
- [71] Zack Whittaker. 2012. *RIM lost \$54 million on four-day global BlackBerry outage*. Retrieved 2024-05-09 from <https://www.zdnet.com/article/rim-lost-54-million-on-four-day-global-blackberry-outage/>

- [72] Phyl Willson, Peter Marshall, Judy Young, and John McCann. 2009. Evaluating the Economic and Social Impact of the National Broadband Network. In *Australasian Conference on Information Systems, ACIS 2009, Melbourne, Australia, December 2-4, 2009*. <https://aisel.aisnet.org/acis2009/27>
- [73] Liudong Xing. 2021. Cascading Failures in Internet of Things: Review and Perspectives on Reliability and Resilience. *IEEE Internet Things J.* 8, 1 (2021), 44–64. <https://doi.org/10.1109/JIOT.2020.3018687>
- [74] Jiaqi Yan and Dong Jin. 2015. VT-Mininet: Virtual-time-enabled Mininet for Scalable and Accurate Software-Define Network Emulation. In *Proceedings of the 1st ACM SIGCOMM Symposium on Software Defined Networking Research, SOSR '15, Santa Clara, California, USA, June 17-18, 2015*, Jennifer Rexford and Amin Vahdat (Eds.). ACM, 27:1–27:7. <https://doi.org/10.1145/2774993.2775012>
- [75] Chao Zhang, Xin Xu, and Hongyan Dui. 2020. Analysis of network cascading failure based on the cluster aggregation in cyber-physical systems. *Reliab. Eng. Syst. Saf.* 202 (2020), 106963. <https://doi.org/10.1016/J.RESS.2020.106963>
- [76] Yibo Zhu, Nanxi Kang, Jiaxin Cao, Albert G. Greenberg, Guohan Lu, Ratul Mahajan, David A. Maltz, Lihua Yuan, Ming Zhang, Ben Y. Zhao, and Haitao Zheng. 2015. Packet-Level Telemetry in Large Datacenter Networks. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication, SIGCOMM 2015, London, United Kingdom, August 17-21, 2015*, Steve Uhlig, Olaf Maennel, Brad Karp, and Jitendra Padhye (Eds.). ACM, 479–491. <https://doi.org/10.1145/2785956.2787483>

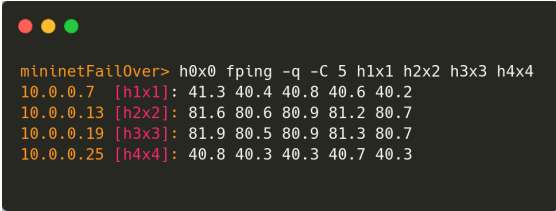
Abbildungsverzeichnis

3.1.	Hamilton Zyklen in einem Torus mit ungeraden Dimensionen. Aus [11]. Zyklus 1: rot; Zyklus 2: schraffiert grau	10
3.2.	Aus [20]. e_t Regeln sind gefüllt. e_l Regeln sind gestrichelt.	10
4.1.	Struktur der Implementierung	19
4.2.	Flussdiagramm Openflow Hamilton Fast Failover	21
4.3.	Flussdiagramm Openflow Low Stretch Verfahren Fast Failover	23
4.4.	Sourceknote (S) in Grün, wenn nach einem Ausfall noch Wege der Länge 2 zu t bestehen. Direkte Nachbarn von t in Blau und Knoten mit einem Weg zu Source in Distanz 2 in Gelb. Für ein Torus $\geq 5 \times 5$	24
4.5.	Flussdiagramm Openflow Augmentation Two-Hop	24
5.1.	Erreichbarkeit unter zufälligen Kantenausfällen. 5×5 Torus.	28
5.2.	Latenzen unter zufälligen Kantenausfällen. 5×5 Torus.	29
5.3.	Erreichbarkeit unter zufälligen Knotenausfällen. 7×7 Torus.	30
5.4.	Latenzen unter zufälligen Knotenausfällen. 7×7 Torus.	30
5.5.	Erreichbarkeit unter Clusterfailure mit $p = 0.9$ und $q \in (0.3, 0.9)$. 7×7 Torus.	32
5.6.	Latenzen unter Clusterfailure mit $p = 0.9$ und $q \in (0.3, 0.9)$. 7×7 Torus.	32
5.7.	Erreichbarkeit unter Clusterfailure Node. $p = 0.9 \wedge q = 0.34$. 7×7 Torus.	33
5.8.	Latenzen unter Clusterfailure Node. $p = 0.9 \wedge q = 0.34$. 7×7 Torus.	33
5.9.	Durchsatz unter zufälligen Kanten Ausfällen 5×5 Torus.; All-to-All; pairs= $ 9 $	35
5.10.	Durchsatz unter zufälligen Knoten Ausfällen 7×7 Torus.; All-to-All; pairs= $ 20 $	36
5.11.	Durchsatz unter Clusterfailure mit $p = 0.9$ und $q \in (0.3, 0.9)$. 7×7 Torus.; All-to-All; pairs= $ 20 $	36
5.12.	Durchsatz unter Clusterfailure mit Node. $p = 0.9 \wedge q = 0.34$. 7×7 Torus.; All-to-All; pairs= $ 20 $	37
5.13.	Durchsatz unter Towards Destination. 5×5 Torus; All to single Destination	37
5.14.	Links: Low Stretch; Mitte: Low Stretch mit Aug; Rechts: angewandtes Failure Pattern	38
5.15.	Erreichbarkeit und zufälligen Kantenausfällen mit Aug. 5×5 Torus.	39
A.1.	Torus 5×5 , Ping von $h0x0$ zu $h1x1$, $h2x2$, $h3x3$, $h4x4$	53
A.2.	Bandbreitenausnutzung von Iperf2 im Simplex und Duplex; $n=5$ (Versuche)	54
A.3.	Scatterplot über Parameter q und p zu einem 7×7 Torus; $n=15$ (Simulation pro Tupel); step=0.005; Aggregation Kantenausfälle per Median	55

A. Anhang Preliminary testing

In diesem Teil des Anhangs sind ungeordnet Ergebnisse des Preliminary testing zusammengefasst, um getroffene Entscheidung innerhalb dieser Arbeit erweitert zu begründen oder Experimente zu präsentieren, die artbedingt doch nicht geeignet waren.

A.1. Latenz



```
mininetFail0ver> h0x0 fping -q -C 5 h1x1 h2x2 h3x3 h4x4
10.0.0.7 [h1x1]: 41.3 40.4 40.8 40.6 40.2
10.0.0.13 [h2x2]: 81.6 80.6 80.9 81.2 80.7
10.0.0.19 [h3x3]: 81.9 80.5 80.9 81.3 80.7
10.0.0.25 [h4x4]: 40.8 40.3 40.3 40.7 40.3
```

Abbildung A.1.: Torus 5x5, Ping von $h0x0$ zu $h1x1$, $h2x2$, $h3x3$, $h4x4$

Wie aus der nicht repräsentativen Stichprobe in Abbildung A.1 zu entnehmen ist, beträgt die maximale Abweichung etwa 1.1 ms. Entsprechend scheint ein Verwerfen des ersten Messwertes überflüssig zu sein, allerdings ist auch hier die Aussagekraft gering. Innerhalb der Rohdaten zeigt sich jedoch ein ähnliches Muster.

A.2. Durchsatz

Innerhalb dieser Sektion wird die Ausnutzung der Bandbreite eines Flows unter Voll-Duplex und als Single Flow analysiert. Da es sich hier nur um das Preliminary Testing handelt, sind die Stichproben relativ klein gewählt und dadurch ist die Generalisierbarkeit eingeschränkt.

Beginnend mit der **Ausnutzung der Bandbreite**: Methodisch wurde das Netzwerk der Größe 5x5 mit dem Low Stretch Verfahren initialisiert und anschließend fünf Stichproben je Flowtyp zwischen $h0x0$ und $h3x3$ erhoben. Als Bandbreitenlimit sind wieder analog 100 Mbit angesetzt. Somit ergeben sich absolute Bandbreitenkapazitäten von 200 Mbit für das Voll-Duplex-Verfahren und 100 Mbit für den Single Flow (Duplikation ist nach Konstruktion des Routingverfahrens ausgeschlossen).

A. Anhang Preliminary testing

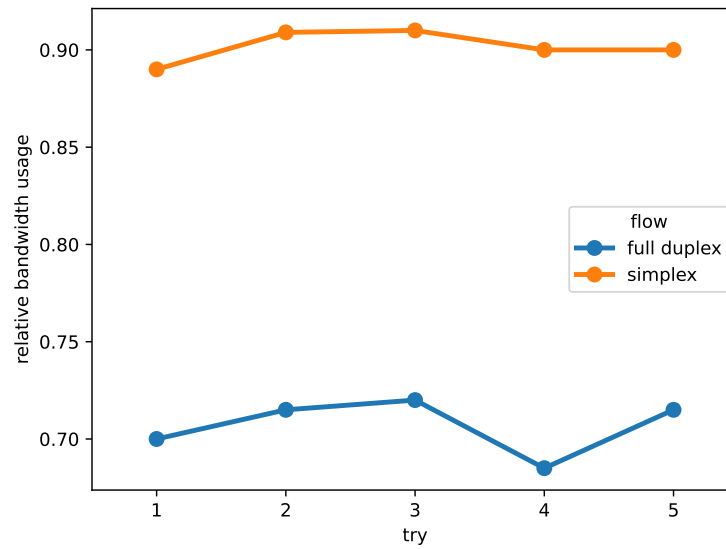


Abbildung A.2.: Bandbreitenausnutzung von Iperf2 im Simplex und Duplex; n=5 (Versuche)

Folgend der Abbildung A.2 ist die relative Bandbreitenausnutzung des Single Flow Verfahrens (im Diagramm als Simplex bezeichnet) durchschnittlich bei 90% und die des Duplex Verfahrens bei 70%. Die Stichproben scheinen ausreichend konsistent zu sein, obwohl im Duplex-Verfahren ein Ausreißer während des vierten Versuchs erkennbar ist. Somit kann das voll duplex Verfahren für die Evaluation gewählt werden.

A.3. Cluster Failure Parametrisierung

Aufgrund der (verhältnismäßig) langwierigen Evaluation (Kapitel 5) kann nur eine geringe Bandbreite der Parameter p und q getestet werden. Folglich wurde ein Skript zur Auswertung verschiedener Kombinationen entworfen, um eine gute Komposition der Parameter zu finden. Zu finden in Abbildung A.3 lässt sich gut oberhalb der »interessante« Bereich ausmachen. Zuerst fiel die Wahl daher auf $p = 0.9$ im Bereich von $x \in (0.3; 0.9)$ um eine gute Progression in jeder Iteration eines Steps zu erreichen. Weiterhin bietet die feste Kombination aus $p = 0.9 \wedge q = 0.34$ einem guten Mittelweg in der Ausdehnung und Variation zwischen den Ausfällen (im letzteren nicht direkt ersichtlich in dem Scatterplot).

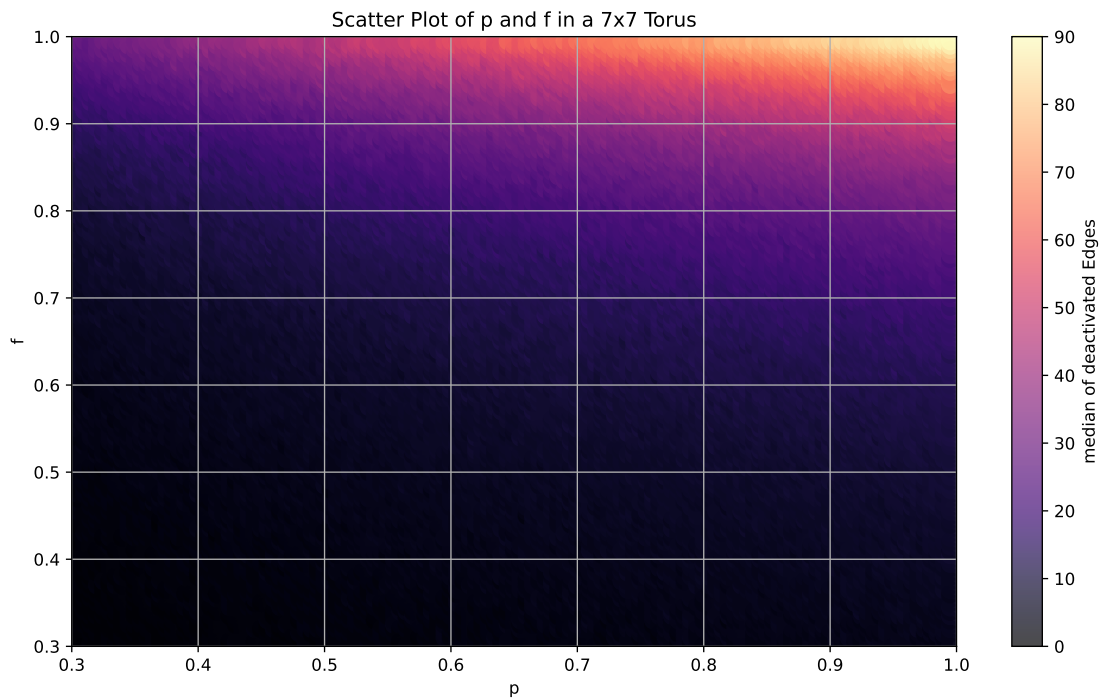


Abbildung A.3.: Scatterplot über Parameter q und p zu einem 7×7 Torus; $n=15$ (Simulation pro Tupel); $\text{step}=0.005$; Aggregation Kantenausfälle per Median

B. Reproduzierbarkeit der Forschungsergebnisse

Reproduzierbarkeit von Forschungsergebnissen ist essenziell, um Zuverlässigkeit und Validität sicherzustellen und eine Weiterentwicklung zu ermöglichen. So führt Bajpai et al.[5] weiter aus, dass einhergehend mit dem Ziel Maßnahmen zu treffen sind, um anderen die Replikation des Experimentellen Set-ups zu erlauben, unter anderem die Veröffentlichung des Source Codes mit ausreichender Dokumentation und die erzeugten Daten. Zu diesem Zweck sind, die Rohdaten und der Source Code des Frameworks auf GitHub veröffentlichen[2]. Eine (ausführliche) Dokumentation erfolgt zeitnah.